



Ôn tập cuối kì ML - Machine Learning

Big data (Trường Đại học Kinh tế Đại học Đà Nẵng)



Scan to open on Studeersnel

ÔN TẬP CUỐI KÌ

Machine Learning

Chương 1:

- Tổng quan về học máy, học máy là gì?
- Lập trình truyền thống và lập trình theo ML ntn?
- Ứng dụng của học máy
- Lịch sử của học máy
- **Phân biệt các loại học máy. Nó dùng để làm gì? Ứng dụng nó ntn**
- **Quy trình triển khai các bước của mô hình machine learning**

Chương 2: Feature Engineering

- Tiền xử lý dữ liệu nó làm cái gì? Mục tiêu của nó.
- Feature Scaling, Feature Encoding(one-hot encoding,... nó như nào vv), Feature Selection, **PCA** (phân tích thành phần chính) làm giảm chiều dữ liệu để có thể hiểu, biểu diễn và đánh giá được các tác động của , Imbalance data
-

Chương 3:

- Decision Tree (entropy, information gain)
- KNN, thuật toán lười
- SVM
- Naives Bayes
- Các phương pháp phân lớp mở rộng
+ Học kết hợp: Bagging, Boosting

Chương 4:

- Clustering
- K-means, k-mode
- Luật kết hợp (Apriori, support confidence)

Chương 5:

- Neural Network
- **CNN rất quan trọng**
- Phân mở rộng về deep learning: nắm rõ về CNN, kiến trúc, các lớp tích chập,..., minh họa cụ thể.
- **Hiểu được kiến trúc quy trình triển khai mô hình DeepLearning phổ biến nhất là CNN**
- **MLP, Gradient Descent, lan truyền ngược**

Chương 1: Giới thiệu về học máy

1. Học máy là gì?

- Là một nhánh của trí tuệ nhân tạo cho phép máy tính học hỏi từ dữ liệu mà không cần lập trình chi tiết.

2. Sự khác nhau giữa lập trình truyền thống và học máy

	Traditional Programing	Machine Learning
Khái niệm	Lập trình viên viết các quy tắc hoặc hướng dẫn rõ ràng để máy tính tuân theo. Các quy tắc này chỉ ra chính	Lập trình viên sẽ đào tạo một mô hình bằng cách sử dụng 1 dữ liệu lớn. Mô hình học các mẫu và mối quan hệ từ dữ

	cách máy tính nên xử lý dữ liệu đầu vào để tạo ra đầu ra mong muốn	liệu, cho phép nó đưa ra dự đoán mà không cần lập trình rõ ràng cho từng khả năng
Phụ thuộc dữ liệu	Ít phụ thuộc vào dữ liệu. Chất lượng đầu ra chủ yếu phụ thuộc vào logic do lập trình viên xác định	Phụ thuộc nhiều vào dữ liệu. Chất lượng dữ liệu đào tạo ảnh hưởng đáng kể đến hiệu suất và độ chính xác của mô hình
Tính linh hoạt và khả năng thích ứng	Tính linh hoạt hạn chế. Những thay đổi trong miền vấn đề đòi hỏi phải cập nhật thủ công mã	Cung cấp khả năng thích ứng cao hơn và các tính hướng mới, đặc biệt nếu mô hình được đào tạo lại bằng dữ liệu cập nhật
Độ phức tạp của vấn đề	Phù hợp nhất với vấn đề có logic rõ ràng, xác định	Xử lý tốt hơn những vấn đề phức tạp mà các mẫu và mối quan hệ không rõ ràng. Vd: Nhận dạng hình ảnh, xử lý ngôn ngữ tự nhiên hoặc phân tích dự đoán
Khả năng dự đoán kết quả	Kết quả có thể dự đoán được cao nếu biết trước các dữ liệu đầu vào và logic	Dự đoán hoặc quyết định dựa trên mô hình học máy đưa ra đôi khi khó diễn giải hơn, đặc biệt là với các mô hình phức tạp như mạng nơ ron sâu

TRADITIONAL PROGRAMMING



MACHINE LEARNING



3. Ứng dụng của Machine Learning

- Nhận diện hình ảnh
 - + Nhận diện khuôn mặt
 - + Phân loại hình ảnh
- Xử lý ngôn ngữ tự nhiên (NLP)
 - + Dịch ngôn ngữ tự nhiên (GG Translate)
 - + Phân tích cảm xúc từ văn bản
 - + Trợ lý ảo (Siri, Alexa,...)
- Dự đoán và phân tích dữ liệu
 - + Dự đoán xu hướng thị trường
 - + Phân tích hành vi khách hàng để cá nhân hóa quảng cáo
- Xe tự động
 - + Hệ thống lái tự động
 - + Nhận diện các vật thể trên đường, điều khiển xe an toàn
- Phát hiện gian lận
 - + Phát hiện giao dịch gian lận trong ngân hàng, thẻ tín dụng
 - + Nhận diện các mẫu hành vi bất thường
- Khuyến nghị sản phẩm
 - + Đề xuất phim, bài hát dựa trên sở thích
 - + Gợi ý sản phẩm mua sắm trên các trang thương mại điện tử
- Y tế
 - + Chẩn đoán bệnh dựa trên dữ liệu y tế
 - + Dự đoán hiệu quả của thuốc và hỗ trợ phát triển dược phẩm mới
- An ninh và giám sát

- + Nhận diện khuôn mặt và giám sát video trong an ninh công cộng
- + Phát hiện xâm nhập mạng và các mối đe dọa an ninh mạng.

4. Các loại học máy

Supervised vs Unsupervised vs Reinforcement Learning

Đặc điểm	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Dữ liệu huấn luyện	Các đặc trưng (features) của dữ liệu, cùng với nhãn tương ứng (hình ảnh và nhãn "mèo" hoặc "chó").	Các đặc trưng của dữ liệu, không có nhãn (tập hợp các bài báo không phân loại).	Trạng thái (state) của môi trường (vị trí của robot trong mê cung, tình hình trên bàn cờ).
Mục tiêu	Dự đoán kết quả (nhãn) cho dữ liệu mới dựa trên các mẫu đã học.	Khám phá cấu trúc ẩn, các mẫu hoặc nhóm trong dữ liệu	Học để tối đa hóa phần thưởng tích lũy từ môi trường
Các kỹ thuật phổ biến	Regression, Decision Trees, KNN Neural Networks...	Clustering (K-Means), Dimensionality Reduction (PCA)	Q-learning, Deep Q-Networks (DQN), Policy Gradients
Ứng dụng	Nhận dạng hình ảnh, lọc thư rác, dự đoán giá nhà...	Phân khúc khách hàng, nén dữ liệu, phát hiện gian lận...	Chơi game, robot điều khiển, tối ưu hóa hệ thống...
Ưu điểm	Hiệu suất cao trên các tác vụ cụ thể, dễ hiểu và diễn giải.	Khám phá kiến thức mới từ dữ liệu, không cần nhãn dữ liệu.	Giải quyết các bài toán phức tạp, thích ứng với môi trường thay đổi.
Nhược điểm	Cần dữ liệu có nhãn, dễ bị overfitting	Khó đánh giá kết quả, kết quả có thể khó diễn giải.	Phức tạp, huấn luyện chậm, cần nhiều dữ liệu tương tác.

11

5. Các khái niệm cơ bản của Machine Learning

- **Supervised learning:** học giám sát trong đó mô hình được huấn luyện dựa trên dữ liệu đã được gắn nhãn để dự đoán nhãn cho dữ liệu mới.
- **Unsupervised Learning:** học không giám sát trong đó mô hình tìm kiếm cấu trúc ẩn trong dữ liệu mà không có nhãn, ví dụ như phân cụm
- **Reinforcement Learning:** học tăng cường, một thuật toán học từ việc tương tác với môi trường và nhận phần thưởng để tối ưu hóa hành động.
- **Overfitting:** Xảy ra khi mô hình học quá chi tiết từ dữ liệu huấn luyện, bao gồm cả nhiễu và các biến động ngẫu nhiên. Điều này dẫn đến mô hình hoạt động rất tốt trên dữ liệu huấn luyện nhưng kém hiệu quả trên dữ liệu mới.
- **Underfitting:** Khi mô hình quá đơn giản, không đủ khả năng nắm bắt các đặc điểm quan trọng trong dữ liệu, dẫn đến hiệu suất kém cả trên dữ liệu huấn luyện lẫn dữ liệu mới.
- **Regularization:** Kỹ thuật để tránh overfitting bằng cách thêm một ràng buộc vào hàm mất mát, phổ biến là L1 và L2 regularization
- **Cross-Validation:** Phương pháp đánh giá hiệu suất của mô hình bằng cách chia dữ liệu thành nhiều phần và huấn luyện mô hình trên các phần khác nhau.
- **Gradient Descent:** Thuật toán tối ưu để tìm cực tiểu của hàm mất mát bằng cách cập nhật các tham số theo hướng ngược lại của gradient
- **Learning rate:** Tham số điều chỉnh tốc độ cập nhật các tham số trong quá trình huấn luyện, ảnh hưởng đến tốc độ và ổn định của quá trình hội tụ
- **Bias-Variance Tradeoff:** sự cân bằng giữa bias (độ lệch) và variance (phương sai), nơi mà mô hình cần duy trì để tránh cả overfitting và underfitting
- **Feature Engineering:** Quá trình tạo ra các đặc trưng mới hoặc chọn lọc các đặc trưng quan trọng từ dữ liệu để cải thiện hiệu suất của mô hình
- **Dimensionality Reduction:** Kỹ thuật giảm số lượng đặc trưng, giúp đơn giản hóa mô hình và giảm nhiễu
- **Hyperparameter Tuning:** Quá trình điều chỉnh các tham số không thể học được trong quá trình huấn luyện, nhằm tối ưu hóa hiệu suất mô hình

- **Confusion Matrix:** Một bảng dùng để đánh giá hiệu suất mô hình phân loại, thể hiện số lượng đúng và sai ở từng lớp
- **Precision and Recall:** Thước đo hiệu suất trong phân loại, precision là tỷ lệ dự đoán đúng trên tất cả các dự đoán, Recall là tỷ lệ dự đoán đúng trên tất cả các trường hợp thực tế
- **ROC-AUC:** Đường cong ROC và diện tích dưới đường cong (AUC) là thước đo đánh giá hiệu suất của mô hình phân loại nhị phân.
- **Neural Networks:** Mạng nơ-ron nhân tạo, mô hình học sâu được thiết kế theo cấu trúc của não bộ con người, có khả năng học các biểu diễn phức tạp từ dữ liệu
- **Bias:** Đề cập đến sự khác biệt giữa giá trị dự đoán của mô hình và giá trị thực tế.
- **Backpropagation:** Thuật toán để tính toán gradient của hàm mất mát đối với các tham số của mạng nơ-ron bằng cách lan truyền ngược từ đầu ra về các lớp trước
- **Variance:** Đề cập đến sự biến động của các dự đoán của mô hình đối với tập dữ liệu khác nhau. Mô hình có variance cao do xu hướng thay đổi mạnh mẽ khi được huấn luyện với các dữ liệu khác nhau, dẫn đến việc mô hình có thể bị overfitting
- **Ensemble Learning:** Kỹ thuật kết hợp nhiều mô hình để cải thiện hiệu suất dự đoán, ví dụ như Random Forest và Gradient Boosting.
- **Bagging:** Phương pháp ensemble, trong đó nhiều mô hình được huấn luyện trên các mẫu ngẫu nhiên khác nhau và kết quả được tổng hợp lại vd: Random Forest
- **Boosting:** Phương pháp ensemble, trong đó các mô hình được hoạt động theo tuần tự, mỗi mô hình sẽ cố gắng sửa lỗi của mô hình trước đó, vd: Gradient Boosting, XGBoost.
- **Loss Function:** Là hàm tính toán giữa dự đoán của mô hình và giá trị thực tế. Nó giúp đánh giá hiệu quả của mô hình và điều chỉnh trọng số trong quá trình huấn luyện để cải thiện độ chính xác. Ví dụ như MSE cho hồi quy
- **SVM:** Thuật toán phân loại tuyến tính hoặc phi tuyến tính, tối ưu việc phân tách hai lớp bằng cách tối đa hóa khoảng cách giữa các điểm dữ liệu và trung tâm cụm
- **PCA:** Kỹ thuật giảm chiều dữ liệu bằng cách tìm các thành phần chính, tức là các hướng trong không gian dữ liệu có phương sai lớn nhất.
- **Bootstrap sampling:** là kỹ thuật lấy mẫu lại với thay thế từ một tập dữ liệu gốc để tạo ra nhiều tập con mẫu, giúp ước lượng sự biến động và độ tin cậy của mô hình/
- **Imbalanced data:** dữ liệu không cân bằng là tình trạng khi các lớp trong tập dữ liệu có số lượng mẫu rất khác nhau. Điều này có thể gây nên khó khăn trong việc huấn luyện mô hình và dẫn đến kết quả dự đoán thiên lệch
- **GANs:** Mạng đối kháng tạo sinh, gồm hai mạng nơ-ron cạnh tranh nhau, một mạng Generator sinh dữ liệu mới, một mạng Discriminator để phân biệt dữ liệu thật giả, giúp cho mô hình tạo ra dữ liệu giống dữ liệu thật

6. Quy trình xây dựng 1 mô hình học máy

Bước 1: Thu thập và tiền xử lý dữ liệu

- Thu thập dữ liệu phù hợp bài toán cần giải quyết
- Làm sạch, tiền xử lý và chuyển đổi dữ liệu

Bước 2: Chia dữ liệu

- Chia dữ liệu thành hai tập: train và test với tỉ lệ 80,20 hoặc 70,30
X_train, X_test, y_test, y_train

Bước 3: Chọn và huấn luyện mô hình

- Chọn một mô hình học máy phù hợp (hồi quy, phân loại)
- Huấn luyện mô hình trên tập huấn luyện để tìm ra các mô hình tối ưu cho mô hình
model.fit(X_train, y_train)

Bước 4: Kiểm tra mô hình

- Sử dụng mô hình đã huấn luyện để dự đoán trên tập kiểm tra
- So sánh dự đoán với kết quả thực tế để đánh giá hiệu suất

`y_pred = model.predict(X_test)`

Bước 5: Đánh giá mô hình

- Đánh giá hiệu quả của mô hình bằng các chỉ số như MAE, MSE, R2, accuracy, F1-score tùy thuộc vào loại bài toán

So sánh `y_test`, `y_pred`

Vd: `MAE = mean_absolute_error(y_test, y_pred)`

Bước 6: Triển khai mô hình

- Triển khai mô hình vào mô hình thực tế để thực hiện các dự đoán hoặc phân loại trên dữ liệu mới

Training and testing a supervised learning model

Chuẩn bị dữ liệu

```
X = data["Waist_cm"]
```

```
y = data["Weight_kg"]
```

Chia dữ liệu thành tập huấn luyện và kiểm tra

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Huấn luyện mô hình

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

Kiểm tra mô hình

```
y_pred = model.predict(X_test)
```

Đánh giá mô hình

```
mae = mean_absolute_error(y_test, y_pred)
```

```
r2 = r2_score(y_test, y_pred)
```

Lưu mô hình

```
model_filename = 'trained_model.pkl'
```

```
joblib.dump(model, model_filename)
```

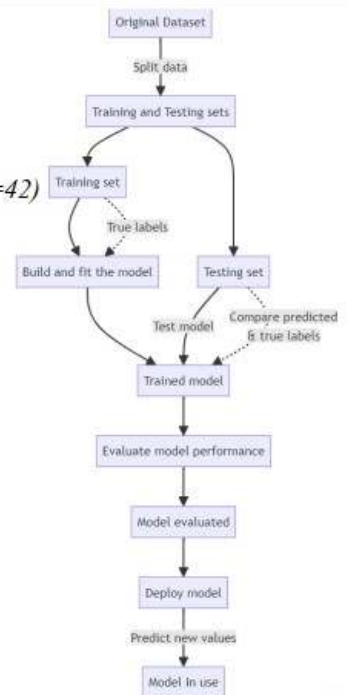
Tải mô hình đã lưu và sử dụng để dự đoán giá trị mới

```
loaded_model = joblib.load(model_filename)
```

Dự đoán với giá trị mới

```
new_waist = np.array([[85]])
```

```
predicted_weight = loaded_model.predict(new_waist)
```



14

7. Các phương pháp đánh giá mô hình học máy

- Training & Testing Process
- K-Fold Cross Validation

Classification Evaluation

- Confusion Matrix
- F- Score
- ROC – AUC

Regression Model Evaluation

- Error
- MAE
- MSE

- R-squared

Underfitting & Overfitting

- Underfitting: xảy ra khi mô hình quá đơn giản và không thể nắm bắt được các mẫu quan trọng trong dữ liệu. Mô hình này có xu hướng bỏ qua các mối quan hệ phức tạp dẫn đến hiệu suất kém trên cả tập huấn luyện và tập kiểm tra.
- Overfitting: Xảy ra khi mô hình quá phức tạp và học thuộc những nhiễu ngẫu nhiên trong dữ liệu huấn luyện. Mô hình này hoạt động rất tốt trên tập huấn luyện nhưng mang lại hiệu quả kém trên tập kiểm tra vì nó không thể tổng quát hoạt tốt trên dữ liệu mới.

Chương 2: Feature Engineering

1. Tổng quan về Feature Engineering

Định nghĩa: Là quá trình tạo ra các đặc trưng mới hoặc biến đổi các đặc trưng hiện có từ dữ liệu thô để cải thiện hiệu suất của mô hình học máy.

Tại sao nó lại quan trọng? Chất lượng của đặc trưng quyết định rất lớn đến hiệu suất của mô hình. Feature Engineering giúp mô hình hiểu dữ liệu tốt hơn bằng cách cung cấp các đặc trưng có liên quan và dễ học hơn, từ đó cải thiện độ hiệu suất của mô hình này.

2. Tiền xử lý dữ liệu là gì?

- Tiền xử lý dữ liệu là bước chuẩn bị dữ liệu thô thành một dạng phù hợp để đưa vào mô hình học máy. Nó bao gồm các thao tác làm sạch, biến đổi, và tổ chức lại dữ liệu để đảm bảo chất lượng và tính nhất quán.

3. Mục tiêu của tiền xử lý dữ liệu

- **Cải thiện chất lượng dữ liệu:** Loại bỏ lỗi, nhiễu và thông tin không cần thiết.
- **Tăng khả năng học của mô hình:** Giảm thiểu các vấn đề như dữ liệu mất cân bằng, giá trị ngoại lai, hoặc dữ liệu không đồng nhất.
- **Chuẩn hóa dữ liệu:** Đưa dữ liệu về cùng định dạng, thang đo, hoặc tỷ lệ để phù hợp với các thuật toán học máy.
- **Tăng tốc độ và hiệu quả xử lý:** Làm gọn dữ liệu, giảm kích thước, hoặc loại bỏ các phần dư thừa.
- **Tối ưu hóa đầu vào cho mô hình:** Đảm bảo dữ liệu đầu vào có ý nghĩa và phù hợp với yêu cầu của thuật toán.

4. Các phương pháp xử lý dữ liệu thiếu

- Bỏ qua (Dropping): xóa hàng, cột có giá trị thiếu
- Điền giá trị (Imputation)
 - + Mean/Median/Mode Imputation: Điền giá trị trung bình, trung vị hoặc tần suất cao nhất của cột vào ô trống. ***Thường sử dụng**
 - + Forward/Backward Fill: Điền giá trị trước đó hoặc sau đó vào ô trống
 - + KNN Imputation: Sử dụng các hàng gần nhất để điền vào ô trống

+ Regression Imputation: Dự đoán giá trị thiếu dựa trên mối quan hệ hồi quy với các biến khác

- Thay thế bằng giá trị đặc biệt (Special Value Imputation): Điền các giá trị -1, 999, “Unknown” vào các ô bị thiếu
- Dự đoán (Predictive Modeling): Sử dụng các mô hình học máy để dự đoán giá trị bị thiếu dựa trên các biến khác
- Multiple Imputation: Tạo ra nhiều bộ dữ liệu hoàn chỉnh bằng cách điền các giá trị thiếu. Sau đó gộp các kết quả phân tích lại để có kết quả cuối cùng.

5. Xử lý ngoại lai

6. Xử lý dữ liệu mất cân bằng

7. Feature Scaling (Chuẩn hóa dữ liệu)

Min-Max Scaling (Normalization)

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Robust Scaler

$$X' = \frac{X - \text{median}}{\text{IQR}}$$

Standardization (Z-score scaling)

$$X' = \frac{X - \mu}{\sigma}$$

MaxAbs Scaling

$$X' = \frac{X}{|X_{\max}|}$$

8. Feature Encoding (Mã hóa dữ liệu)

- One-Hot Encoding: Mã hóa từng giá trị thành một cột nhị phân riêng biệt
- Dummy Encoding: Tương tự One-hot, nhưng loại bỏ một cột để tránh đa cộng tuyến hoàn hảo (thường dùng làm biến gốc hay baseline)
- Label Encoding: Mỗi giá trị phân loại được ánh xạ sang một số nguyên khác nhau
- Target Encoding: Thay thế các giá trị phân loại bằng trung bình của biến mục tiêu cho từng nhóm phân loại
- Frequency Encoding: Mỗi giá trị phân loại được thay thế bằng tần số xuất hiện của nó trong tập dữ liệu.
- Binary Encoding: Mã hóa giá trị phân loại thành chuỗi nhị phân

9. Feature Selection (Lựa chọn đặc trưng)

Phương pháp lọc (Filter Methods)

- **Kỹ thuật dựa trên thống kê:** Sử dụng các chỉ số thống kê để đánh giá độ liên quan của đặc trưng với biến mục tiêu:
 - **Correlation Coefficient:** Đánh giá mối quan hệ tuyến tính giữa đặc trưng và biến mục tiêu.
 - **Chi-squared test:** Đánh giá sự phụ thuộc giữa đặc trưng và biến mục tiêu
 - **ANOVA:** Đánh giá sự khác biệt giữa các nhóm

Phương pháp Wrapper

- Sử dụng mô hình học máy để đánh giá sự kết hợp của các đặc trưng

- **Forward Selection:** Bắt đầu với một đặc trưng và thêm từ từ các đặc trưng khác nếu chúng cải thiện hiệu suất mô hình.
- **Backward Elimination:** Bắt đầu với tất cả các đặc trưng và loại bỏ các đặc trưng không cần thiết
- **Recursive Feature Elimination (RFE):** Loại bỏ lần lượt các đặc trưng ít quan trọng nhất dựa trên mô hình học máy

Forward Selection

Input: Dataset with n features and target variable Y .

Output: Optimal subset of features.

1. Initialize:

$S_{\text{selected}} = []$ # Tập hợp các đặc trưng được chọn (ban đầu rỗng)

$S_{\text{remaining}} = [X_1, X_2, \dots, X_n]$ # Tập hợp các đặc trưng chưa được chọn

$\text{Best_score} = 0$ # Điểm số tốt nhất hiện tại (ví dụ: Accuracy, F1-score, MSE,...)

2. Repeat until stopping criterion is met:

2.1. For each feature X in $S_{\text{remaining}}$:

- Build a model using features $S_{\text{selected}} + X$.
- Evaluate the model performance (e.g., cross-validation).
- Record the performance score.

2.2. Select the feature X_{best} with the highest performance score.

2.3. If the performance score with $X_{\text{best}} > \text{Best_score}$:

- Add X_{best} to S_{selected} .
- Remove X_{best} from $S_{\text{remaining}}$.
- Update $\text{Best_score} = \text{performance score of } X_{\text{best}}$.

2.4. Else:

- Stop the algorithm (no further improvement).

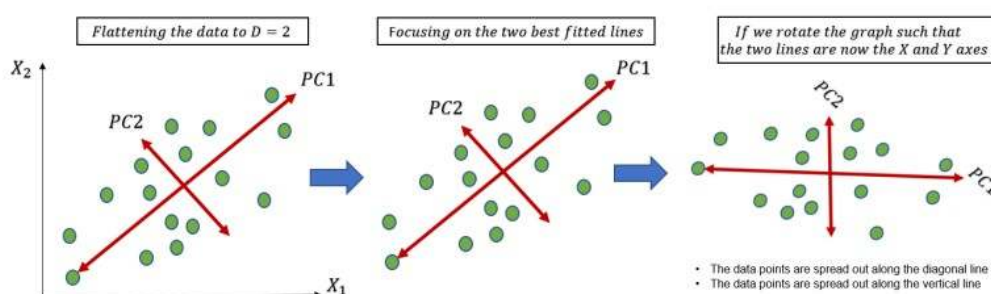
3. Return the final subset of features in S_{selected} .

10. Dimensionality Reduction (Giảm chiều dữ liệu)

- Là quá trình giảm số lượng biến (features) trong một tập dữ liệu mà vẫn giữ được những thông tin quan trọng nhất. Kỹ thuật này giúp tăng hiệu suất mô hình, giảm thời gian xử lý, tránh overfitting.
- Có hai nhóm chính:
 - **Feature Selection:** chọn ra những đặc trưng quan trọng nhất (Filter Methods, Wrapped Methods, Embedded Methods)
 - **Feature Extraction:** Biến đổi dữ liệu để giảm số chiều nhưng vẫn giữ được thông tin quan trọng (PCA, LDA)

11. PCA

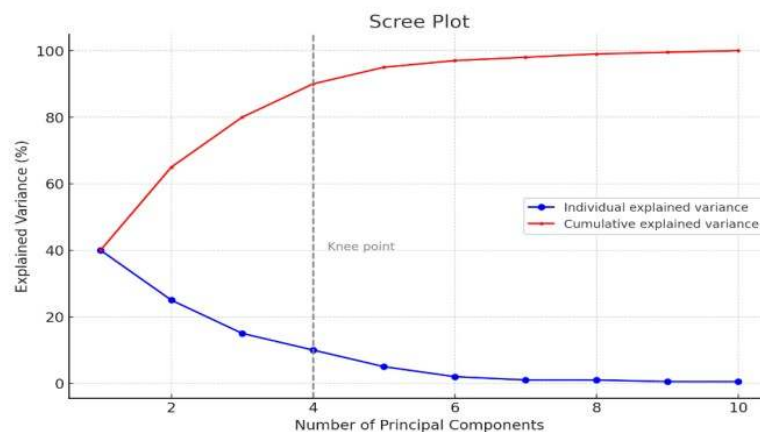
- Giảm số lượng đặc trưng bằng cách biến đổi các biến ban đầu thành các thành phần chính đại diện cho phần lớn thông tin dữ liệu.



Quy trình thực hiện PCA

1. **Chuẩn hóa dữ liệu:** Chuẩn hóa dữ liệu để các biến có trung bình $=0$ và độ lệch chuẩn $=1$
2. **Tính ma trận hiệp phương sai (Covariance Matrix):** Tính ma trận hiệp phương sai, tính toán các giá trị riêng và vector riêng
 - Vector riêng: Xác định hướng của các thành phần chính
 - Giá trị riêng: Đại diện cho lượng phương sai giải thích được mỗi thành phần chính
3. **Sắp xếp các thành phần chính theo thứ tự giảm dần của phương sai:** Sắp xếp các thành phần chính theo giá trị riêng của chúng, giá trị lớn nhất trước.
4. **Chọn số lượng thành phần chính:** Quyết định số lượng thành phần chính cần giữ lại trên tổng phương sai giải thích (Scree Plot)
5. **Chuyển đổi dữ liệu:** Chuyển đổi dữ liệu ban đầu sang không gian mới của các thành phần chính được chọn

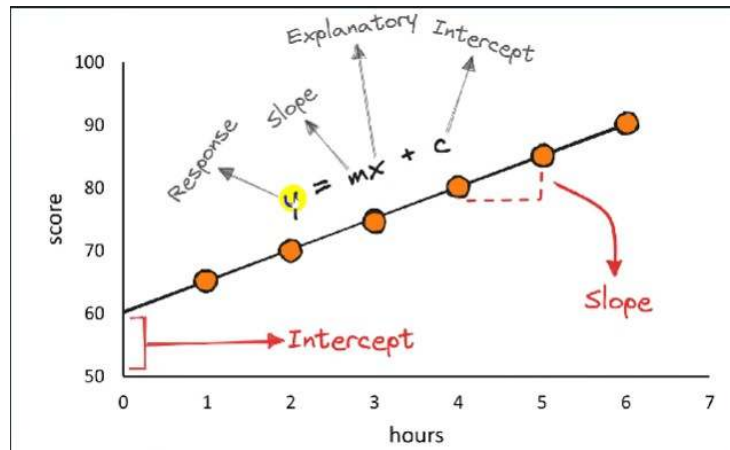
Chọn số thành phần chính với scree plot



10 Thuật toán học máy phổ biến

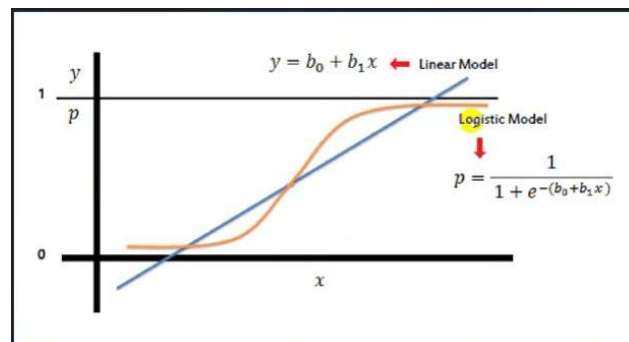
1. Linear Regression

- **Cách thức hoạt động:** Tìm mối quan hệ tuyến tính giữa biến đầu vào và biến đầu ra bằng cách tìm đường thẳng phù hợp nhất, tối ưu hóa hàm mất mát dựa trên bình phương sai số
- **Mục đích:** Dự đoán giá trị liên tục
- **Điểm mạnh:** Đơn giản, dễ hiểu
- **Điểm yếu:** Hiệu quả kém với dữ liệu phi tuyến
- **Ứng dụng:** Dự đoán giá nhà



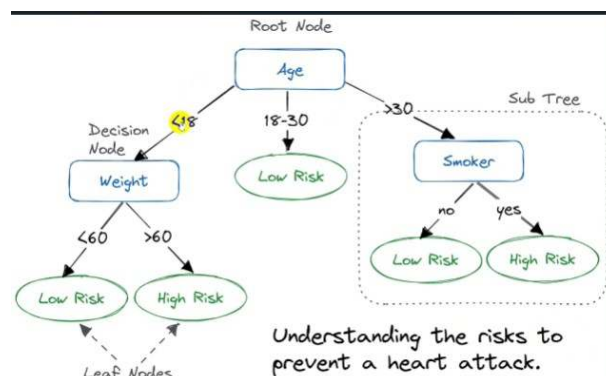
2. Logistic Regression

- **Cách thức hoạt động:** Sử dụng hàm logistics (sigmoid) để dự đoán xác suất của một kết quả nhị phân
- **Mục đích:** Phân loại nhị phân
- **Điểm mạnh:** Dễ triển khai, diễn giải rõ ràng
- **Điểm yếu:** Không mạnh với các quan hệ phi tuyến
- **Ứng dụng:** Phát hiện gian lận, phân loại email spam



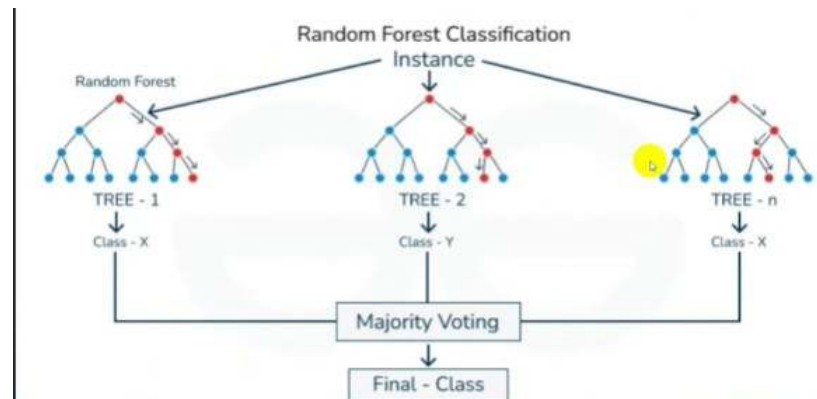
3. Decision Tree

- **Cách thức hoạt động:** Phân chia dữ liệu bằng các điều kiện if...and...then dựa trên các đặc trưng, dẫn đến các quyết định cuối cùng của các nút lá
- **Mục đích:** Phân loại và hồi quy
- **Điểm mạnh:** Diễn giải dễ dàng, không cần tiền xử lý dữ liệu
- **Điểm yếu:** Dễ bị overfitting
- **Ứng dụng:** Chuẩn đoán y khoa, phân loại sản phẩm



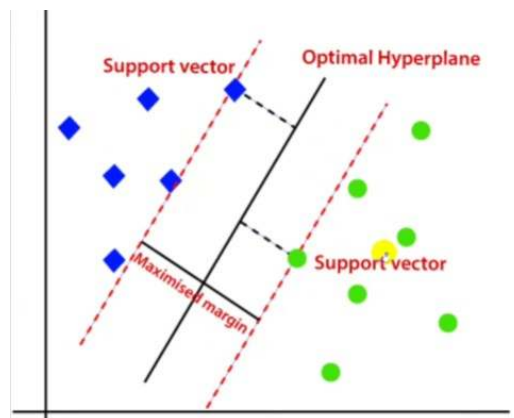
4. Random Forest

- **Cách thức hoạt động:** Kết hợp nhiều cây quyết định bằng phương pháp bagging, mỗi cây được xây dựng từ một mẫu ngẫu nhiên của dữ liệu và kết quả cuối cùng được quyết định bằng cách lấy trung bình hoặc bỏ phiếu đa số.
- **Mục đích:** Phân loại và hồi quy
- **Điểm mạnh:** Giảm overfitting hiệu quả cao
- **Điểm yếu:** Chậm với tập dữ liệu lớn
- **Ứng dụng:** Dự đoán tài chính, phân tích thị trường



5. SVM (Support Vector Machine)

- **Cách thức hoạt động:** Tìm một siêu phẳng tối ưu để phân tách các lớp trong không gian đặc trưng, sử dụng các vector hỗ trợ để xác định biên giới phân tách
- **Mục đích:** Phân loại và hồi quy
- **Điểm mạnh:** Hiệu quả với dữ liệu phi tuyến
- **Điểm yếu:** Khó diễn giải, tiêu tốn nhiều tài nguyên
- **Ứng dụng:** Nhận diện hình ảnh, phát hiện gian lận

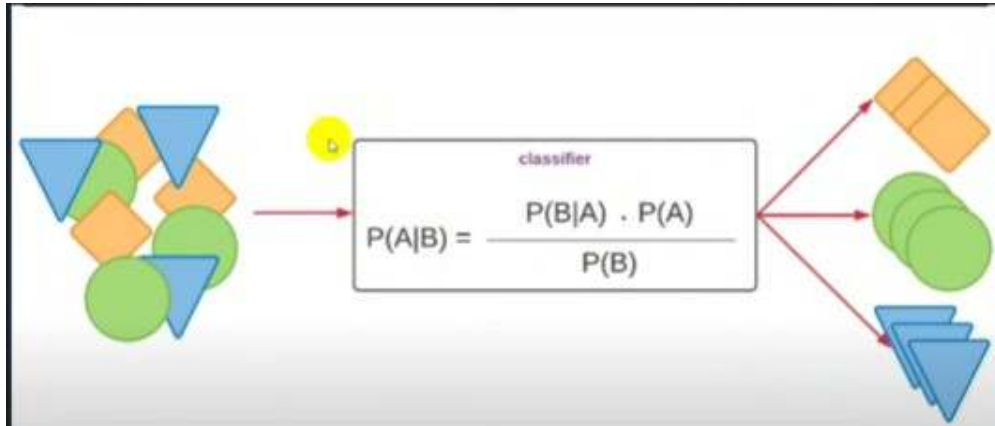


6. KNN

- **Cách thức hoạt động:** Dự đoán nhãn của một điểm dữ liệu mới dựa trên nhãn của k điểm dữ liệu gần nhất trong không gian đặc trưng, xác định bằng bỏ phiếu đa số hoặc trung bình.
- **Mục đích:** Phân loại và hồi quy
- **Điểm mạnh:** Đơn giản, không cần huấn luyện
- **Điểm yếu:** Chậm với dữ liệu lớn
- **Ứng dụng:** Nhận dạng chữ viết tay, nhận diện mẫu

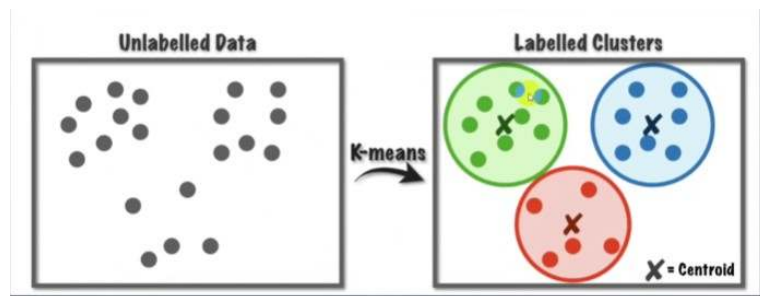
7. Naive Bayes

- **Cách thức hoạt động:** Dựa trên định lý Bayes và giả định rằng các đặc trưng độc lập với nhau, tính xác suất của các lớp dựa trên xác suất có điều kiện của các đặc trưng và chọn lớp có xác suất cao nhất.
- **Mục đích:** Phân loại
- **Điểm mạnh:** Nhanh, hiệu quả với dữ liệu lớn
- **Điểm yếu:** Giả định độc lập mạnh mẽ
- **Ứng dụng:** Phân loại văn bản, phát hiện spam



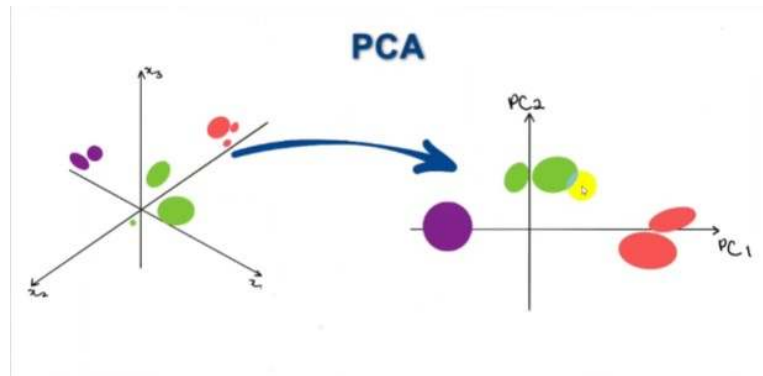
8. K-Means Clustering

- **Cách thức hoạt động:** Phân chia các điểm dữ liệu vào k cụm bằng cách tối thiểu hóa tổng bình phương khoảng cách giữa các điểm dữ liệu và trung tâm cụm, cập nhật trung tâm cụm lặp đi lặp lại cho đến khi hội tụ
- **Mục đích:** Phân nhóm học không giám sát
- **Điểm mạnh:** Đơn giản, hiệu quả
- **Điểm yếu:** Phụ thuộc vào lựa chọn K, không đảm bảo tìm được tối ưu toàn cục
- **Ứng dụng:** Phân khúc khách hàng, thị trường, phân tích hành vi



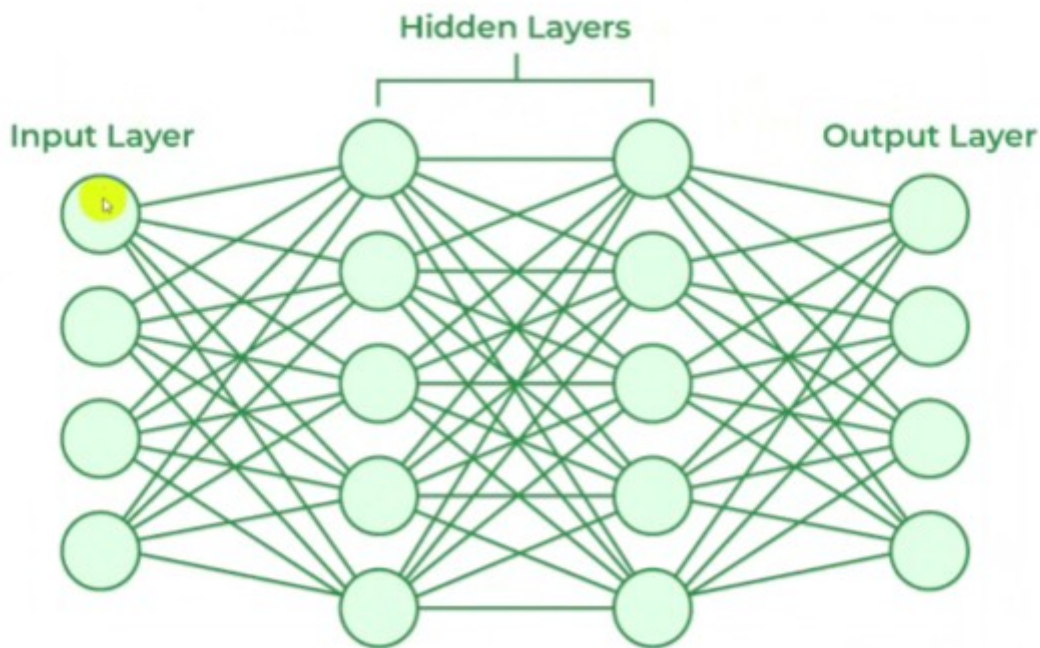
9. PCA (Principle Component Analysis)

- **Cách thức hoạt động:** Tìm các trục chính để mô tả tốt nhất dữ liệu bằng góc cách tính toán các vector riêng và giá trị riêng, sử dụng các trục này để biến đổi dữ liệu về không gian mới với số chiều giảm
- **Mục đích:** Giảm chiều dữ liệu
- **Điểm mạnh:** Loại bỏ dư thừa, tăng tốc độ
- **Điểm yếu:** Mất thông tin, khó diễn giải
- **Ứng dụng:** Tìm mẫu, Nén dữ liệu, xử lý hình ảnh



10. ANN (Artificial Neural Network)

- **Cách thức hoạt động:** Gồm các tầng nơ-ron (input, output, hidden) liên kết với nhau, mỗi nơ-ron nhận input, thực hiện phép tính với các trọng số và bias, rồi áp dụng hàm kích hoạt. Mạng được huấn luyện bằng cách tối ưu hóa hàm mất mát thông qua các phương pháp như gradient descent và backpropagation.
- **Mục đích:** Phân loại, hồi quy, học sâu
- **Điểm mạnh:** Linh hoạt, hiệu quả với dữ liệu lớn và phức tạp
- **Điểm yếu:** Cần nhiều dữ liệu, khó diễn giải
- **Ứng dụng:** Nhận diện giọng nói, hình ảnh, xử lý ngôn ngữ tự nhiên



Chương 3: Decision Tree

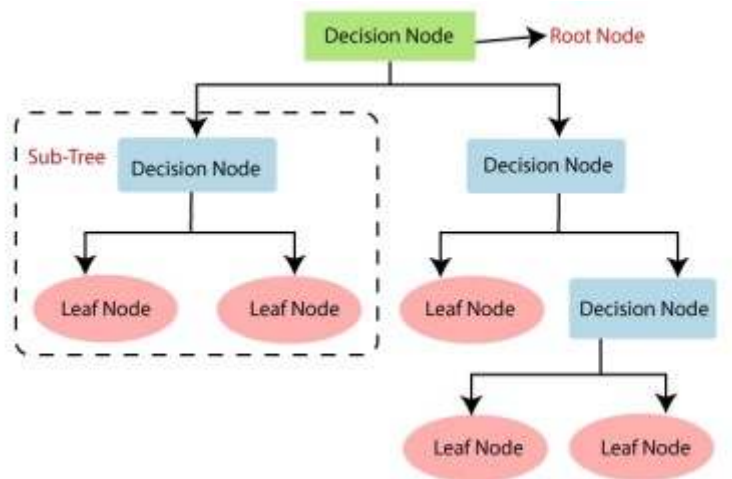
1. Cây quyết định là gì?

- Cây quyết định là mô hình học máy có cấu trúc dạng cây, dùng để phân loại hoặc dự đoán.
- Gồm hai loại:
 - Cây phân loại (Classification Tree)
 - Cây hồi quy (Regression Tree)

2. Cấu trúc của cây quyết định

- Nút gốc (Root Node): Đây là nút bắt đầu của cây

- Nút quyết định (Decision Node): Đại diện cho các điểm kiểm tra hoặc quyết định
- Nút lá (Leaf Node): Đây là các nút cuối cùng của cây. Mỗi nút đại diện cho một nhãn dự đoán.
- Nhánh (Branch): Kết nối các nút trong cây.
- Cây con (Subtree): là một phần của cây quyết định, bắt đầu từ một nút bất kỳ và bao gồm tất cả các nút con và nhánh bên dưới nút đó

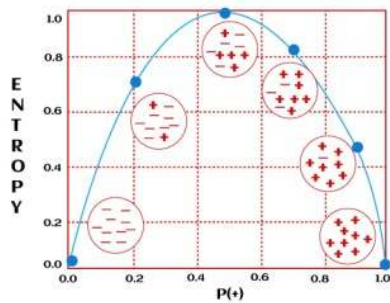


3. Các thuật toán xây dựng cây quyết định

- **ID3**: Thuật toán cơ bản dùng độ lợi thông tin (information gain) để xây dựng cây quyết định, chỉ áp dụng cho bài toán phân loại.
- **C4.5**: Là phiên bản cải tiến của ID3, hỗ trợ cả dữ liệu liên tục và xử lý giá trị thiếu, sử dụng tỷ lệ thông tin (information gain ratio) để tránh thiên vị khi phân chia.
- **CART**: Xây dựng cây nhị phân, có thể áp dụng cho cả phân loại và hồi quy, sử dụng Gini impurity cho phân loại và MSE cho hồi quy để làm tiêu chí phân chia.
- **Random Forest**: Kết hợp nhiều cây quyết định để cải thiện độ chính xác và giảm overfitting
- **XGBoost**: Là phương pháp tăng cường dựa trên cây quyết định, đặc biệt hiệu quả trong các bài toán học máy phức tạp. Tối ưu hóa quá trình huấn luyện để tăng tốc độ và độ chính xác.
- **LightGBM**: Một thuật toán tăng cường gradient dựa trên cây quyết định, được tối ưu hóa để xử lý các tập dữ liệu lớn, với khả năng huấn luyện nhanh hơn và hiệu quả hơn XGBoost
- **CatBoost**: Một thuật toán boosting dựa trên cây quyết định, đặc biệt hiệu quả khi làm việc với dữ liệu phân loại.

4. Entropy & Information Gain

- Entropy là thước đo mức độ không chắc chắn hoặc hỗn loạn của một tập dữ liệu
- Entropy thấp: Khi tập dữ liệu hầu hết thuộc về một lớp (ít hỗn loạn)
- Entropy cao: Khi các lớp có tỷ lệ gần bằng nhau (rất hỗn loạn)



$$\text{Entropy}(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

- S : Tập dữ liệu.
- c : Số lớp (class) trong tập dữ liệu.
- p_i : Tỷ lệ (xác suất) của lớp thứ i trong tập dữ liệu S .

- Information Gain của một thuộc tính A đối với tập dữ liệu S là sự giảm của Entropy khi phân tách dữ liệu S theo thuộc tính A
- Thuộc tính nào có IG cao nhất sẽ được chọn để phân chia dữ liệu.

$$\text{IG}(S, A) = \text{Entropy}(S) - \sum_{v \in V} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)$$

- S : Tập dữ liệu ban đầu.
- A : Thuộc tính được dùng để chia.
- V : Các giá trị có thể có của thuộc tính A
- S_v : Tập con của S tương ứng với giá trị v của thuộc tính A .
- $|S_v|$: Số lượng phần tử trong tập con S_v .
- $|S|$: Số lượng phần tử trong tập dữ liệu ban đầu.

5. Thuật toán xây dựng cây quyết định ID3 (Vd đọc slide)

- Được phát triển bởi Ross Quinlan vào năm 1986. ID3 sử dụng chỉ số Entropy và Information Gain để chọn thuộc tính tối ưu và phân chia dữ liệu một cách đệ quy

Bước 1: Tính toán Entropy

Bước 2: Tính toán Information Gain cho từng thuộc tính

Bước 3: Chọn thuộc tính có Information Gain cao nhất để phân chia dữ liệu

Bước 4: Phân chia dữ liệu: Sau khi chọn thuộc tính phân chia, dữ liệu được chia thành các tập con dựa trên các giá trị của thuộc tính đó

Bước 5: Lặp lại quá trình cho các tập con: Thực hiện quá trình từ bước 1 đến bước 4 cho từng tập con một cách đệ quy, cho đến khi tất cả dữ liệu tổng tập con thuộc cùng một lớp (entropy=0) hoặc không còn thuộc tính nào để phân chia

6. Cắt tỉa cây quyết định

- Kỹ thuật cắt tỉa (pruning) cây quyết định là phương pháp được sử dụng để giảm thiểu độ phức tạp của mô hình và ngăn ngừa hiện tượng overfitting bằng cách loại bỏ các nhánh không có giá trị dự đoán cao, giúp tăng độ tổng quát của mô hình.

Các phương pháp cắt tỉa cây quyết định:

- Cắt tỉa trước (Pre-pruning hay Early Stopping)
 - Cây sẽ dừng phát triển tại một mức độ nào đó dựa trên các tiêu chí nhất định, chẳng hạn như giới hạn độ sâu của cây hoặc số lượng mẫu tối thiểu cần thiết để tách nhánh
 - Ngăn chặn cây phát triển quá phức tạp ngay từ đầu, do đó tiết kiệm tài nguyên tính toán nhưng có nguy cơ dừng quá sớm và bỏ qua các mẫu quan trọng.
- Cắt tỉa sau (Post-pruning):
 - Phát hiện cây đầy đủ trước và sau đó cắt tỉa, thường cho kết quả tốt hơn vì cây đã khám phá hết các mqh trong dữ liệu trước khi loại bỏ các nhánh không quan trọng.
 - Một cách tiếp cận phổ biến là hợp nhất các nhánh nhỏ hoặc loại bỏ những nhánh mà sự giảm thiểu lỗi không đáng kể.

Các tiêu chí cắt tỉa:

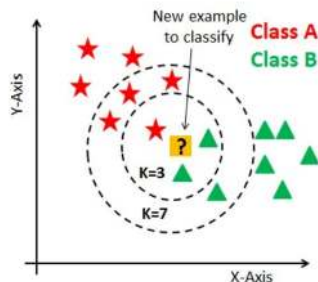
- Chi phí lỗi: Phương pháp này tìm lại sự cân bằng giữa kích thước của cây và hiệu suất dự đoán của cây bằng cách đánh giá mức độ cải thiện của mỗi nhánh so với chi phí tăng thêm.
- Giảm độ nhiễu: Các nhánh được loại bỏ nếu chúng không làm giảm sai số của mô hình khi kiểm tra trên Validation set.

Thuật toán KNN

1. KNN là gì

- Là một thuật toán phân loại và hồi quy trong học máy, thuộc nhóm **lazy learning** (học lười biếng).
- Thay vì học một mô hình từ dữ liệu, KNN lưu trữ toàn bộ dữ liệu huấn luyện và đưa ra dự đoán dựa trên các điểm gần nhất trong không gian đặc trưng.

2. Nguyên lý hoạt động của KNN



- Chọn số láng giềng gần
- Tính khoảng cách từ đối tượng cần xếp lớp đến
- Sắp xếp tăng dần theo khoảng cách
- Chọn ra k đối tượng đầu tiên
- Xếp đối tượng vào lớp có tần suất nhiều nhất trong k đối tượng đó

3. Các phương pháp đo lường khoảng cách tương đồng

- Euclidean
- Manhattan
- Minkowski
- Chebyshev
- Cosine
- Jaccard
- Hamming

- Weight voting

4. Ưu điểm

- Dễ hiểu, dễ triển khai.
- Hiệu quả với các tập dữ liệu nhỏ, không phức tạp.
- Không yêu cầu tham số hóa dữ liệu.

5. Nhược điểm

- Nhạy cảm với nhiễu khi K nhỏ
- KNN là một thuật toán mà mọi tính toán đều nằm ở khâu test. Trong đó việc tính khoảng cách tới từng điểm dữ liệu trong training set sẽ tốn rất nhiều thời gian, đặc biệt là với các cơ sở dữ liệu có số chiều lớn và có nhiều điểm dữ liệu. Với K càng lớn thì độ phức tạp cũng sẽ tăng lên. Ngoài ra, việc lưu toàn bộ dữ liệu trong bộ nhớ cũng ảnh hưởng tới hiệu năng của KNN.

6. Ứng dụng

1. Phân loại hình ảnh

KNN được sử dụng để phân loại hình ảnh dựa trên các đặc trưng như màu sắc, hình dạng hoặc kết cấu.

- **Phân loại chữ viết tay:**

- Dữ liệu: Hình ảnh các chữ số viết tay (MNIST dataset).
- Quy trình:
 1. Biểu diễn hình ảnh dưới dạng vector pixel.
 2. Tính khoảng cách giữa hình ảnh cần phân loại và các hình ảnh trong tập huấn luyện.
 3. Dựa vào K chữ số gần nhất để dự đoán nhãn.
- Kết quả: Phân loại hình ảnh đầu vào thành các chữ số từ 0 đến 9.

2. Phân loại văn bản

KNN được dùng để phân loại tài liệu hoặc email dựa trên nội dung.

- **Phát hiện email spam:**

- Dữ liệu: Email chứa nội dung (subject, body).
- Quy trình:
 1. Chuyển email thành vector từ khóa (bag-of-words hoặc TF-IDF).
 2. Tính khoảng cách giữa email cần phân loại và các email đã được gán nhãn (spam hoặc không spam).
 3. Nhãn của email được dự đoán dựa trên nhãn của K email gần nhất.
- Kết quả: Email được phân loại là spam hoặc không spam.

3. Hệ thống gợi ý sản phẩm

KNN được sử dụng trong các hệ thống gợi ý sản phẩm dựa trên hành vi của người dùng.

- **Gợi ý phim:**

- Dữ liệu: Lịch sử xem phim của người dùng, xếp hạng phim.
- Quy trình:
 1. Tính khoảng cách giữa người dùng cần gợi ý với các người dùng khác dựa trên sở thích phim.
 2. Tìm K người dùng có sở thích gần nhất.
 3. Gợi ý phim dựa trên các phim mà K người dùng này đã xem hoặc đánh giá cao.

- Kết quả: Hệ thống gợi ý phim có khả năng người dùng sẽ yêu thích.

4. Chẩn đoán bệnh

KNN được sử dụng trong y học để hỗ trợ chẩn đoán bệnh dựa trên các thông số sinh học.

- **Dự đoán nguy cơ mắc tiểu đường:**

- Dữ liệu: Các chỉ số sức khỏe như lượng đường huyết, huyết áp, chỉ số BMI.
- Quy trình:
 1. Tính khoảng cách giữa bệnh nhân cần dự đoán với các hồ sơ bệnh nhân trong cơ sở dữ liệu.
 2. Dựa vào K bệnh nhân gần nhất, xác định nguy cơ mắc tiểu đường.
- Kết quả: Chẩn đoán xem bệnh nhân có khả năng mắc bệnh tiểu đường hay không.

5. Phân khúc khách hàng (Customer Segmentation)

KNN giúp phân loại khách hàng dựa trên hành vi mua sắm, độ tuổi, hoặc thu nhập.

- **Phân nhóm khách hàng trong bán lẻ:**

- Dữ liệu: Thông tin khách hàng bao gồm số tiền chi tiêu, tần suất mua sắm.
- Quy trình:
 1. Tính khoảng cách giữa khách hàng cần phân loại và các khách hàng khác.
 2. Gán khách hàng này vào nhóm của K khách hàng gần nhất.
- Kết quả: Khách hàng được phân nhóm (ví dụ: nhóm VIP, nhóm khách hàng tiềm năng).

6. Phát hiện gian lận

KNN được dùng để phát hiện hành vi bất thường trong giao dịch tài chính.

- **Phát hiện giao dịch thẻ tín dụng gian lận:**

- Dữ liệu: Thông tin giao dịch như số tiền, địa điểm, thời gian.
- Quy trình:
 1. Tính khoảng cách giữa giao dịch mới và các giao dịch lịch sử.
 2. Nếu giao dịch mới khác biệt đáng kể so với K giao dịch gần nhất, đánh dấu là gian lận.
- Kết quả: Xác định giao dịch gian lận để cảnh báo.

7. Dự đoán giá trị bất động sản

KNN có thể dự đoán giá trị của một căn nhà dựa trên các đặc trưng như diện tích, vị trí, số phòng.

- **Dự đoán giá nhà:**

- Dữ liệu: Thông tin các căn nhà, bao gồm diện tích, vị trí, số phòng ngủ.
- Quy trình:
 1. Tính khoảng cách giữa căn nhà cần dự đoán và các căn nhà đã biết giá trị.
 2. Dự đoán giá bằng cách tính trung bình giá trị của K căn nhà gần nhất.
- Kết quả: Dự đoán giá trị của căn nhà đầu vào.

8. Phát hiện bất thường trong dữ liệu (Anomaly Detection)

KNN được dùng để phát hiện các điểm dữ liệu bất thường trong tập dữ liệu.

- **Phát hiện bất thường trong mạng lưới máy tính:**

- Dữ liệu: Lưu lượng mạng, thời gian kết nối, số lượng gói tin.
- Quy trình:

1. Tính khoảng cách giữa một mẫu lưu lượng mạng và các mẫu khác.
2. Nếu mẫu này cách quá xa K mẫu gần nhất, đánh dấu là bất thường.
- Kết quả: Phát hiện các hoạt động bất thường hoặc tấn công mạng.

9. Phân cụm dữ liệu không gắn nhãn

KNN có thể được sử dụng để hỗ trợ phân cụm trong các bài toán semi-supervised learning.

- **Phân cụm khách hàng tiềm năng:**

- Dữ liệu: Hành vi mua sắm của khách hàng chưa được gắn nhãn.
- Quy trình:
 1. Sử dụng KNN để xác định các cụm khách hàng dựa trên khoảng cách.
 2. Áp dụng nhãn cho từng cụm dựa trên khách hàng đã được gắn nhãn (nếu có).
- Kết quả: Phân cụm khách hàng chưa rõ ràng vào các nhóm cụ thể.

7. Thuật toán lười biếng (Lazy Learning)

- Thuật toán lười biếng không thực hiện việc học từ dữ liệu trong giai đoạn huấn luyện. Thay vào đó, dữ liệu được lưu trữ và chỉ xử lý khi có yêu cầu dự đoán.
- Đặc điểm:
 - **Không mô hình hóa:** Không tạo ra mô hình chung từ dữ liệu huấn luyện
 - **Chậm khi dự đoán:** Thời gian xử lý chủ yếu tập trung vào giai đoạn dự đoán

Linear Regression

1. Cách thức hoạt động

Linear Regression là một thuật toán học máy thuộc nhóm Supervised Learning, hoạt động bằng cách tìm mối quan hệ tuyến tính giữa biến đầu vào (features) và biến đầu ra (target). Thuật toán cố gắng tìm một đường thẳng trong không gian đặc trưng (feature space) sao cho tổng bình phương sai số (Sum of Squared Errors - SSE) giữa các giá trị thực tế và giá trị dự đoán là nhỏ nhất.

- **Biểu thức cơ bản:**

Đường thẳng dự đoán có dạng:

$$y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

Trong đó:

- y : Biến mục tiêu cần dự đoán.
- $x_1, x_2, \dots, x_n$: Các biến đầu vào.
- w_0 : Intercept (điểm giao với trục y).
- $w_1, w_2, \dots, w_n$: Trọng số của các biến đầu vào, thể hiện mức độ ảnh hưởng của từng biến đối với biến mục tiêu.
- **Hàm mất mát (Loss Function):**
Linear Regression sử dụng Mean Squared Error (MSE) để đo lường sai số:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Thuật toán tối ưu trọng số (ww) để giảm thiểu MSE bằng cách sử dụng các phương pháp như Gradient Descent.

2. Mục đích

Linear Regression được sử dụng để dự đoán giá trị liên tục, chẳng hạn như giá cả, nhiệt độ, doanh thu, hoặc bất kỳ đại lượng nào có thể được đo lường theo các giá trị số.

• Ví dụ minh họa:

Nếu bạn muốn dự đoán giá nhà dựa trên các yếu tố như diện tích, số phòng ngủ, vị trí và năm xây dựng, Linear Regression có thể được áp dụng. Mô hình sẽ xác định một phương trình tuyến tính dựa trên các yếu tố này để dự đoán giá nhà.

3. Điểm mạnh

- Đơn giản và dễ hiểu: Linear Regression có công thức đơn giản và kết quả dễ dàng được diễn giải, rất phù hợp cho các trường hợp yêu cầu tính minh bạch.
- Hiệu quả với dữ liệu tuyến tính: Khi dữ liệu đầu vào có mối quan hệ tuyến tính mạnh với đầu ra, Linear Regression thường hoạt động tốt.
- Tốc độ cao: Quá trình huấn luyện nhanh chóng ngay cả với tập dữ liệu lớn, đặc biệt khi số lượng biến đầu vào không quá nhiều.

4. Điểm yếu

- Hiệu quả kém với dữ liệu phi tuyến: Linear Regression không thể nắm bắt được các quan hệ phức tạp và phi tuyến. Trong các trường hợp này, cần chuyển sang các thuật toán như Polynomial Regression hoặc sử dụng các thuật toán phi tuyến khác như Random Forest, Neural Networks.
- Nhạy cảm với outliers: Một vài giá trị ngoại lai (outliers) có thể ảnh hưởng mạnh đến kết quả dự đoán, do đó cần làm sạch dữ liệu kỹ lưỡng trước khi áp dụng.
- Giả định hạn chế: Linear Regression giả định mối quan hệ tuyến tính giữa các biến và phân phối của sai số phải chuẩn (normal distribution), điều này không phải lúc nào cũng đúng trong thực tế.

5. Ứng dụng phổ biến

Linear Regression có nhiều ứng dụng thực tiễn trong các lĩnh vực khác nhau, bao gồm:

- Dự đoán giá nhà: Sử dụng các đặc trưng như diện tích, số phòng ngủ, vị trí để dự đoán giá nhà.
- Phân tích tài chính: Dự đoán lợi nhuận hoặc chi phí dựa trên dữ liệu lịch sử.
- Y tế: Dự đoán thời gian hồi phục của bệnh nhân dựa trên các yếu tố như tuổi tác, mức độ nghiêm trọng của bệnh, phương pháp điều trị.
- Marketing: Dự đoán doanh số bán hàng dựa trên ngân sách quảng cáo và các yếu tố khác như mùa vụ.
- Kinh tế học: Phân tích và dự đoán các chỉ số kinh tế như tỷ lệ thất nghiệp, GDP, hoặc giá trị đầu tư.

Ví dụ chi tiết: Dự đoán doanh số bán hàng

- Dữ liệu: Một công ty có dữ liệu về chi tiêu cho quảng cáo trên TV, radio, và báo chí.
- Mục tiêu: Dự đoán doanh số bán hàng.
- Mô hình:

$$Sales = w_0 + w_1(TV) + w_2(Radio) + w_3(Newspaper)$$

Quy trình:

1. Thu thập và tiền xử lý dữ liệu.
2. Xây dựng mô hình Linear Regression và tối ưu hóa trọng số.
3. Đánh giá mô hình qua các chỉ số như R^2 , MSE.

7. Tổng kết

Linear Regression là một công cụ mạnh mẽ nhưng cũng cần được sử dụng đúng hoàn cảnh. Nếu dữ liệu tuân theo các giả định cơ bản, nó cung cấp một cách tiếp cận đơn giản và hiệu quả. Tuy nhiên, trong các bài toán phức tạp hơn, việc áp dụng các thuật toán khác có thể mang lại kết quả tốt hơn.

Logistic Regression

1. Cách thức hoạt động

Logistic Regression là một thuật toán Supervised Learning được sử dụng chủ yếu cho bài toán phân loại, đặc biệt là phân loại nhị phân (binary classification). Thuật toán dựa trên hồi quy tuyến tính nhưng sử dụng hàm logistic (hay còn gọi là sigmoid) để dự đoán xác suất của các lớp.

- **Hàm logistic (sigmoid):**

Biến đầu ra của Logistic Regression nằm trong khoảng $[0, 1]$, nhờ vào hàm logistic được định nghĩa như sau:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Trong đó:

- $z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$: Tổng trọng số của các biến đầu vào.
- **Dự đoán phân loại:**
Sau khi tính toán xác suất (PP) với hàm sigmoid, Logistic Regression sử dụng một ngưỡng (threshold) để đưa ra quyết định phân loại:
 - Nếu $P \geq 0.5$: Dự đoán thuộc lớp 1.
 - Nếu $P < 0.5$: Dự đoán thuộc lớp 0.
- **Hàm mất mát (Loss Function):**
Logistic Regression sử dụng hàm Cross-Entropy Loss để tối ưu hóa mô hình:

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Trong đó:

- y_i : Giá trị thực tế (0 hoặc 1).
- $\hat{y}_i$: Xác suất dự đoán của lớp 1.

2. Mục đích

Mục tiêu chính của Logistic Regression là phân loại nhị phân, tức là phân loại một đối tượng thuộc vào một trong hai nhóm, dựa trên các đặc điểm đầu vào. Ngoài ra, Logistic Regression còn cung cấp xác suất của từng lớp, hữu ích trong các ứng dụng yêu cầu giải thích chi tiết.

3. Điểm mạnh

- Dễ triển khai: Logistic Regression có tính toán đơn giản và dễ áp dụng.
- Khả năng diễn giải: Các trọng số (w) có thể được diễn giải để hiểu mức độ ảnh hưởng của từng biến đầu vào lên kết quả.
- Nhanh và hiệu quả: Thời gian huấn luyện ngắn, phù hợp với các tập dữ liệu nhỏ đến trung bình.
- Dự đoán xác suất: Khả năng dự đoán xác suất giúp ứng dụng trong các bài toán ra quyết định dựa trên rủi ro.

4. Điểm yếu

- Hạn chế với dữ liệu phi tuyến: Logistic Regression không thể nắm bắt các quan hệ phi tuyến giữa biến đầu vào và biến mục tiêu.
- Nhạy cảm với outliers: Giá trị ngoại lai có thể ảnh hưởng mạnh đến kết quả.
- Phụ thuộc vào giả định tuyến tính: Logistic Regression giả định rằng mối quan hệ giữa các biến đầu vào và log-odds của biến mục tiêu là tuyến tính.

5. Ứng dụng phổ biến

Logistic Regression được ứng dụng rộng rãi trong nhiều lĩnh vực, bao gồm:

- Phát hiện gian lận: Dự đoán xem một giao dịch có phải là gian lận hay không dựa trên các đặc điểm như giá trị giao dịch, vị trí, và thời gian.
- Phân loại email spam: Xác định xem một email là spam hay không dựa trên nội dung và các thông tin tiêu đề email.
- Y tế: Dự đoán khả năng mắc bệnh dựa trên các yếu tố như tuổi, cân nặng, tiền sử bệnh lý.

- Phân tích rủi ro tài chính: Dự đoán khả năng khách hàng không trả được nợ dựa trên lịch sử tín dụng, thu nhập, và số tiền vay.
- Hệ thống đề xuất: Dự đoán khả năng người dùng sẽ nhấp vào một quảng cáo dựa trên lịch sử hành vi và sở thích.

Ví dụ chi tiết: Phát hiện gian lận

- Dữ liệu: Một ngân hàng có dữ liệu giao dịch, bao gồm số tiền, thời gian, vị trí, và loại giao dịch.
- Mục tiêu: Phát hiện xem giao dịch có phải là gian lận hay không.
- Mô hình:
Logistic Regression xác định xác suất giao dịch là gian lận (P), sau đó sử dụng ngưỡng (ví dụ: $P \geq 0.7$) để đánh dấu giao dịch cần kiểm tra thêm.

6. Tổng kết

Logistic Regression là một mô hình mạnh mẽ và đơn giản cho các bài toán phân loại nhị phân. Dù không phù hợp với các bài toán phức tạp, nó vẫn được ưu tiên trong nhiều tình huống nhờ khả năng diễn giải rõ ràng và hiệu quả tính toán. Trong thực tế, Logistic Regression thường là bước khởi đầu để kiểm tra tính chất dữ liệu trước khi sử dụng các mô hình phân loại phức tạp hơn.

Random Forest

1. Cách thức hoạt động

Random Forest là một thuật toán học máy thuộc nhóm ensemble learning. Nó sử dụng nhiều cây quyết định (decision trees) và kết hợp chúng để cải thiện độ chính xác và giảm overfitting.

Quy trình hoạt động chính:

1. Tạo nhiều mẫu dữ liệu con (bootstrapping):
 - Từ tập dữ liệu ban đầu, tạo ra nhiều tập dữ liệu ngẫu nhiên (mẫu con) bằng cách chọn ngẫu nhiên dữ liệu với hoàn lại.
2. Xây dựng cây quyết định cho mỗi mẫu con:
 - Với mỗi mẫu con, xây dựng một cây quyết định dựa trên việc chọn ngẫu nhiên các đặc trưng (features).
 - Các cây này thường chỉ được xây sâu một mức độ nhất định để tránh overfitting.
3. Kết hợp kết quả:
 - Đối với phân loại: Dùng phương pháp bỏ phiếu đa số (majority voting) từ các cây.
 - Đối với hồi quy: Tính trung bình giá trị dự đoán từ các cây.

Lợi ích của việc kết hợp nhiều cây:

- Các cây riêng lẻ có thể yếu hoặc mắc lỗi, nhưng khi kết hợp, lỗi sẽ được giảm đáng kể do tính đa dạng của các cây.

- Phương pháp bagging (Bootstrap Aggregating) làm giảm phương sai và tăng độ chính xác của mô hình.

2. Mục đích sử dụng

Random Forest có thể dùng cho cả hai loại bài toán:

- Phân loại: Dự đoán nhãn (labels) của các lớp. Ví dụ: Phân loại email là spam hay không spam.
- Hồi quy: Dự đoán giá trị liên tục. Ví dụ: Dự đoán giá nhà hoặc doanh thu.

3. Điểm mạnh

1. Giảm overfitting:
 - Vì Random Forest sử dụng nhiều cây với dữ liệu và đặc trưng ngẫu nhiên, nó không phụ thuộc quá mức vào một tập dữ liệu cụ thể. Điều này giúp mô hình tổng quát tốt hơn so với các cây quyết định riêng lẻ.
2. Hiệu quả với dữ liệu phức tạp:
 - Random Forest hoạt động tốt với dữ liệu phi tuyến tính, dữ liệu có nhiều biến tương tác, hoặc chứa nhiều đặc trưng không quan trọng.
3. Tự động xử lý missing values:
 - Trong một số trường hợp, Random Forest vẫn có thể hoạt động tốt khi dữ liệu thiếu giá trị.
4. Tầm quan trọng của đặc trưng (Feature Importance):
 - Random Forest có thể cung cấp thông tin về mức độ quan trọng của từng đặc trưng đối với dự đoán, giúp ích trong việc chọn lọc đặc trưng (feature selection).

4. Điểm yếu

1. Chậm và tiêu tốn tài nguyên với tập dữ liệu lớn:
 - Với số lượng cây lớn và dữ liệu lớn, việc xây dựng và dự đoán trở nên chậm và tiêu tốn nhiều bộ nhớ.
2. Khó diễn giải:
 - Mặc dù các cây quyết định riêng lẻ dễ hiểu, nhưng khi kết hợp hàng trăm hoặc hàng nghìn cây, mô hình trở nên khó giải thích hơn.
3. Nhạy cảm với số lượng cây và hyperparameter:
 - Hiệu quả của Random Forest phụ thuộc vào số lượng cây ($n_{\text{estimators}}$), số đặc trưng được chọn (max_features), và chiều sâu tối đa của cây (max_depth).

5. Ứng dụng thực tế

1. Dự đoán tài chính:
 - Ví dụ: Phân loại khách hàng tiềm năng, dự đoán giá cổ phiếu, hoặc đánh giá rủi ro tín dụng.
2. Phân tích thị trường:
 - Ví dụ: Xác định phân khúc khách hàng hoặc dự đoán xu hướng mua sắm.

3. Y tế:
 - Dự đoán bệnh dựa trên triệu chứng hoặc dữ liệu gen.
4. Khoa học môi trường:
 - Dự đoán thời tiết, phân loại thảm thực vật, hoặc đánh giá chất lượng không khí.
5. Thương mại điện tử:
 - Dự đoán hành vi khách hàng, gợi ý sản phẩm.
6. Phát hiện gian lận:
 - Nhận diện các giao dịch bất thường hoặc hành vi gian lận trong tài chính hoặc bảo hiểm.

6. So sánh với các thuật toán khác

Tiêu chí	Random Forest	Decision Tree	XGBoost
Overfitting	Giảm overfitting	Dễ bị overfitting	Ít bị overfitting
Tốc độ huấn luyện	Chậm hơn Decision Tree	Nhanh hơn	Chậm hơn RF
Diễn giải	Khó diễn giải	Dễ diễn giải	Khó hơn RF
Hiệu quả	Cao, đặc biệt với dữ liệu phức tạp	Phụ thuộc vào cấu trúc cây	Thường cao hơn RF

SVM (Support Vector Machine)

1. Cách thức hoạt động

Support Vector Machine (SVM) là một thuật toán học máy mạnh mẽ, dùng để giải quyết cả bài toán phân loại và hồi quy. Nó hoạt động bằng cách tìm một siêu phẳng (hyperplane) tối ưu để phân tách các lớp dữ liệu trong không gian đặc trưng (feature space).

Quy trình hoạt động:

1. Xác định siêu phẳng tối ưu:

- SVM tìm siêu phẳng phân tách các lớp sao cho khoảng cách (margin) giữa siêu phẳng và các điểm dữ liệu gần nhất (các vector hỗ trợ – support vectors) được tối đa hóa.

2. Sử dụng kernel trick:

- Khi dữ liệu không tuyến tính, SVM sử dụng kernel function để ánh xạ dữ liệu sang một không gian đặc trưng cao hơn, nơi các lớp có thể được phân tách tuyến tính.
- Các kernel phổ biến:
 - Linear Kernel: Dùng khi dữ liệu có thể phân tách tuyến tính.
 - Polynomial Kernel: Ánh xạ không gian với quan hệ đa thức.
 - RBF (Radial Basis Function): Ánh xạ dữ liệu thành không gian đa chiều với Gaussian function.

3. Biên giới quyết định (decision boundary):

- Biên giới được xác định dựa trên các vector hỗ trợ, tạo thành vùng phân tách giữa các lớp.

Trong bài toán hồi quy (SVR - Support Vector Regression):

- SVM tìm một đường hồi quy tối ưu sao cho tất cả các điểm dữ liệu nằm trong một ngưỡng sai số epsilon (ϵ).

2. Mục đích sử dụng

- Phân loại:
 - Dùng để xác định nhãn của dữ liệu trong các bài toán như phân loại văn bản, nhận diện hình ảnh.
- Hồi quy:
 - Dự đoán các giá trị liên tục trong các bài toán như dự đoán giá cổ phiếu hoặc dự đoán nhu cầu thị trường.

3. Điểm mạnh

1. Hiệu quả với dữ liệu phi tuyến:
 - Kernel trick giúp SVM xử lý tốt dữ liệu phức tạp với mối quan hệ phi tuyến.
2. Hiệu suất cao với dữ liệu nhỏ:
 - SVM hoạt động rất hiệu quả khi số lượng dữ liệu không quá lớn nhưng độ phức tạp cao.
3. Chống overfitting:
 - Khi được tối ưu hóa tốt (đặc biệt với kernel và các tham số như CC), SVM ít bị overfitting.
4. Hỗ trợ dữ liệu nhiều chiều:
 - SVM hoạt động tốt với không gian đặc trưng nhiều chiều và dữ liệu nhiều chiều.

4. Điểm yếu

1. Khó diễn giải:
 - Đặc biệt khi sử dụng các kernel phức tạp, mô hình trở nên khó hiểu hơn so với các thuật toán đơn giản như Decision Tree.
2. Tiêu tốn tài nguyên:
 - Khi dữ liệu lớn hoặc nhiều đặc trưng, SVM có thể rất chậm, cả về huấn luyện lẫn dự đoán.
3. Nhạy cảm với dữ liệu không cân bằng:
 - SVM có thể hoạt động kém nếu một lớp có quá nhiều dữ liệu so với lớp còn lại.
4. Lựa chọn kernel và tham số phức tạp:
 - Hiệu quả của SVM phụ thuộc nhiều vào việc chọn đúng kernel và các tham số như CC (điều chỉnh biên độ) và γ (trong RBF kernel).

5. Ứng dụng thực tế

1. Nhận diện hình ảnh:
 - Ví dụ: Phân loại ảnh mèo và chó, nhận diện khuôn mặt trong các hệ thống bảo mật.
2. Phát hiện gian lận:
 - Ví dụ: Xác định giao dịch đáng ngờ trong hệ thống ngân hàng.
3. Phân tích văn bản:
 - Ví dụ: Phân loại email spam, phân tích cảm xúc từ dữ liệu mạng xã hội.
4. Y tế:
 - Ví dụ: Dự đoán bệnh từ dữ liệu y học như MRI hoặc ECG.
5. Phân loại sinh học:
 - Ví dụ: Phân loại chuỗi gen, protein hoặc dự đoán cấu trúc phân tử.
6. Thương mại điện tử:
 - Ví dụ: Phân tích hành vi khách hàng và gợi ý sản phẩm.

6. So sánh với các thuật toán khác

Tiêu chí	SVM	Random Forest	Regression
Dữ liệu phi tuyến	Xử lý tốt nhờ kernel	Phụ thuộc vào số lượng cây	Không hiệu quả
Diễn giải	Khó với kernel phức tạp	Khó với nhiều cây	Rõ ràng và dễ hiểu
Hiệu quả dữ liệu nhỏ	Cao	Tốt	Tốt
Tốc độ huấn luyện	Chậm với dữ liệu lớn	Chậm hơn SVM	Nhanh hơn nhiều

Naive Bayes

Naive Bayes – Phân tích chi tiết

1. Cách thức hoạt động

Naive Bayes là một nhóm thuật toán phân loại dựa trên Định lý Bayes với giả định rằng các đặc trưng (features) trong dữ liệu độc lập với nhau. Đây là lý do thuật toán được gọi là "Naive" (ngây thơ).

Định lý Bayes được biểu diễn như sau:

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

- $P(C|X)$: Xác suất mà X thuộc vào lớp C (posterior probability).
- $P(X|C)$: Xác suất có điều kiện của X với điều kiện thuộc lớp C (likelihood).
- $P(C)$: Xác suất tiên nghiệm của lớp C (prior probability).
- $P(X)$: Xác suất tổng quát của dữ liệu X (evidence).

Cách hoạt động của thuật toán:

1. Tính xác suất tiên nghiệm:

- Dựa trên phân phối dữ liệu, tính toán xác suất tiên nghiệm $P(C)$ cho từng lớp.

2. Tính xác suất có điều kiện:

- Dựa trên từng đặc trưng $x_{i,j}$, tính $P(x_i|C)P(x_j|C)$ cho từng lớp C .

3. Giả định độc lập:

- Các đặc trưng $x_1, x_2, \dots, x_n$ được coi là độc lập, do đó:

$$P(X|C) = P(x_1|C) \cdot P(x_2|C) \cdot \dots \cdot P(x_n|C)$$

4. Dự đoán lớp:

- Kết hợp các xác suất trên và chọn lớp C với xác suất $P(C|X)$ cao nhất.

Các biến thể phổ biến của Naive Bayes dựa trên loại dữ liệu:

- Gaussian Naive Bayes: Dùng cho dữ liệu liên tục (theo phân phối Gaussian).
- Multinomial Naive Bayes: Dùng cho dữ liệu rời rạc, đặc biệt phù hợp với phân loại văn bản.
- Bernoulli Naive Bayes: Dùng cho dữ liệu nhị phân (0 hoặc 1).

2. Mục đích sử dụng

Naive Bayes được thiết kế chủ yếu cho các bài toán phân loại. Một số ứng dụng phổ biến bao gồm:

- Phân loại nhãn (label classification).
- Xác định lớp của dữ liệu dựa trên đặc trưng.
- Phân loại văn bản, như phân loại tài liệu thành các danh mục hoặc xác định ý nghĩa cảm xúc.

3. Điểm mạnh

1. Nhanh và hiệu quả:
 - Naive Bayes rất nhanh trong quá trình huấn luyện và dự đoán, đặc biệt khi làm việc với dữ liệu lớn.
2. Hiệu quả với dữ liệu nhiều chiều:
 - Vì giả định các đặc trưng độc lập, thuật toán không bị ảnh hưởng bởi số lượng chiều dữ liệu.
3. Hiệu quả với dữ liệu nhỏ:
 - Cần ít dữ liệu để huấn luyện, nhưng vẫn đạt hiệu quả cao nếu dữ liệu phù hợp với giả định độc lập.
4. Xử lý tốt dữ liệu mất cân bằng:
 - Phù hợp cho các tập dữ liệu có tỷ lệ không đồng đều giữa các lớp.
5. Dễ triển khai:
 - Thuật toán đơn giản, dễ hiểu và dễ triển khai.

4. Điểm yếu

1. Giả định độc lập mạnh mẽ:

- Giả định rằng các đặc trưng là độc lập với nhau thường không đúng trong thực tế, dẫn đến sai số.
- 2. Không linh hoạt với dữ liệu phi tuyến:
 - Không hiệu quả khi dữ liệu có mối quan hệ phi tuyến phức tạp giữa các đặc trưng.
- 3. Nhạy cảm với dữ liệu không phù hợp:
 - Nếu xác suất có điều kiện gần bằng 0 cho một đặc trưng nào đó, toàn bộ xác suất dự đoán sẽ bằng 0 (có thể khắc phục bằng smoothing, như Laplace Smoothing).
- 4. Phụ thuộc vào phân phối xác suất:
 - Hiệu suất giảm nếu dữ liệu không phù hợp với phân phối giả định, như Gaussian hoặc Multinomial.

5. Ứng dụng thực tế

1. Phân loại văn bản:
 - Phân loại email: Dự đoán email spam hoặc không spam.
 - Phân loại tài liệu: Xếp tài liệu vào các danh mục, như chính trị, khoa học, nghệ thuật.
2. Phát hiện gian lận:
 - Xác định các giao dịch đáng ngờ hoặc lừa đảo trong ngân hàng hoặc thương mại điện tử.
3. Xác định ý nghĩa cảm xúc:
 - Phân tích cảm xúc từ dữ liệu mạng xã hội, đánh giá sản phẩm.
4. Hệ thống gợi ý:
 - Sử dụng để phân loại sở thích người dùng dựa trên dữ liệu lịch sử.
5. Y tế:
 - Chẩn đoán bệnh dựa trên các triệu chứng hoặc phân loại tế bào ung thư từ dữ liệu y học.
6. Tài chính:
 - Dự đoán rủi ro tín dụng hoặc phân tích xu hướng thị trường.

6. So sánh với các thuật toán khác

Tiêu chí	Naive Bayes	Logistic Regression	SVM	Random Forest
Tốc độ	Rất nhanh	Nhanh	Chậm hơn	Chậm với dữ liệu lớn
Hiệu quả dữ liệu nhỏ	Cao	Trung bình	Cao	Trung bình
Xử lý phi tuyến	Không hiệu quả	Hạn chế	Tốt nhờ kernel trick	Tốt
Dữ liệu lớn	Tốt	Tốt	Kém với nhiều đặc trưng	Tốt nhưng tiêu tốn tài nguyên
Diễn giải	Dễ hiểu	Dễ hiểu	Khó với kernel phức tạp	Khó với nhiều cây

PCA (Principle Component Analysis)

1. Cách thức hoạt động

Principal Component Analysis (PCA) là một kỹ thuật giảm chiều dữ liệu bằng cách tìm các hướng (trục chính) quan trọng nhất trong dữ liệu, sao cho khi biểu diễn dữ liệu trong không gian mới, lượng thông tin được bảo toàn nhiều nhất.

(Principal Component Analysis - PCA)

PCA là một kỹ thuật giúp giảm số lượng biến (features) trong một bộ dữ liệu bằng cách chuyển đổi chúng thành một tập hợp các biến mới, gọi là các thành phần chính (principal components).

Giảm chiều dữ liệu:

- Khi bạn có một bộ dữ liệu với rất nhiều biến (features), việc phân tích và trực quan hóa có thể trở nên phức tạp.
- PCA giúp giảm số lượng biến này mà vẫn giữ được phần lớn thông tin gốc.

Các thành phần chính:

- PCA chuyển đổi các biến gốc thành các thành phần chính. Mỗi thành phần chính là một tổ hợp tuyến tính của các biến gốc.
- Các thành phần chính được sắp xếp theo thứ tự giảm dần về mức độ đóng góp vào tổng phương sai (variance) của dữ liệu.

Giảm chiều dữ liệu:

- Khi bạn có một bộ dữ liệu với rất nhiều biến (features), việc phân tích và trực quan hóa có thể trở nên phức tạp.
- PCA giúp giảm số lượng biến này mà vẫn giữ được phần lớn thông tin gốc.

Các thành phần chính:

- PCA chuyển đổi các biến gốc thành các thành phần chính. Mỗi thành phần chính là một tổ hợp tuyến tính của các biến gốc.
- Các thành phần chính được sắp xếp theo thứ tự giảm dần về mức độ đóng góp vào tổng phương sai (variance) của dữ liệu.
- Thành phần chính đầu tiên (PC1) là hướng (vector) mà dữ liệu có phương sai lớn nhất.

Phương sai lớn nhất: PC1 là hướng mà các điểm dữ liệu trải rộng nhất, tức là nếu bạn chiếu tất cả các điểm dữ liệu lên PC1, chúng sẽ có sự phân tán (biến thiên) lớn nhất. Phương sai là thước đo của sự phân tán, và PC1 nắm bắt được nhiều biến thiên nhất trong dữ liệu so với bất kỳ hướng nào khác.

- Thành phần chính thứ hai (PC2) có hướng vuông góc với PC1, có phương sai lớn thứ hai, và cứ tiếp tục như vậy.

Ứng dụng của PCA:

Giảm chiều dữ liệu (Dimensionality Reduction): Giảm số lượng biến trong bộ dữ liệu mà vẫn giữ lại phần lớn thông tin ban đầu

Trực quan hóa dữ liệu (Data Visualization): Trực quan hóa dữ liệu phức tạp nhiều chiều trong không gian 2D hoặc 3D

Khử nhiễu (Noise Reduction): Loại bỏ các thành phần chứa ít thông tin hoặc nhiễu trong dữ liệu

Nén dữ liệu (Data Compression): Nén dữ liệu bằng cách giữ lại các thành phần chính có phương sai lớn và bỏ qua các thành phần khác

Các bước cơ bản của PCA:

1. Chuẩn hóa dữ liệu:

- Đảm bảo rằng các đặc trưng (features) có cùng đơn vị bằng cách chuẩn hóa dữ liệu (mean = 0, variance = 1).

2. Tính ma trận hiệp phương sai (Covariance Matrix):

- Tính mối quan hệ giữa các đặc trưng để hiểu mức độ tương quan giữa chúng.

3. Tính toán vector riêng (Eigenvectors) và giá trị riêng (Eigenvalues):

- Vector riêng xác định hướng của các trục chính (principal components).
- Giá trị riêng đo lường độ lớn (mức độ quan trọng) của mỗi trục chính.

4. Sắp xếp và chọn các trục chính:

- Sắp xếp các trục chính theo thứ tự giảm dần của giá trị riêng.
- Chọn các trục chính đầu tiên (kích thước không gian mới) sao cho tỷ lệ phương sai được giữ lại đáp ứng yêu cầu (ví dụ, giữ lại 95% phương sai).

5. Biến đổi dữ liệu:

- Dự án dữ liệu ban đầu lên không gian của các trục chính để thu được dữ liệu giảm chiều.

2. Mục đích sử dụng

PCA được sử dụng chủ yếu để:

- Giảm chiều dữ liệu:
 - Giảm số lượng đặc trưng trong dữ liệu, giúp tăng tốc độ xử lý và giảm yêu cầu tài nguyên.
- Loại bỏ nhiễu:
 - Loại bỏ thông tin không quan trọng, giữ lại các thành phần chính.
- Phân tích mẫu:
 - Tìm hiểu các mẫu (patterns) hoặc cấu trúc trong dữ liệu phức tạp.

3. Điểm mạnh

1. Loại bỏ dư thừa:
 - PCA loại bỏ sự phụ thuộc giữa các đặc trưng, giảm thông tin dư thừa.
2. Tăng tốc độ xử lý:
 - Giảm chiều dữ liệu giúp giảm kích thước tính toán trong các mô hình học máy.
3. Phù hợp với dữ liệu có tương quan cao:
 - Rất hiệu quả khi các đặc trưng có sự tương quan, vì PCA tận dụng mối quan hệ này để tái cấu trúc dữ liệu.
4. Dễ triển khai:
 - PCA là một phương pháp toán học đã được chuẩn hóa và có sẵn trong nhiều thư viện, như scikit-learn.

4. Điểm yếu

1. Mất thông tin:
 - Khi giảm chiều, một số thông tin có thể bị mất, đặc biệt khi không giữ đủ số lượng trục chính.
2. Khó diễn giải:
 - Các trục chính là tổ hợp tuyến tính của các đặc trưng ban đầu, khiến việc hiểu ý nghĩa của chúng trở nên khó khăn.
3. Không phù hợp với dữ liệu phi tuyến:

- PCA chỉ xử lý tốt dữ liệu có mối quan hệ tuyến tính. Đối với dữ liệu phi tuyến, cần dùng các biến thể như Kernel PCA.
- 4. Yêu cầu chuẩn hóa dữ liệu:
 - Nếu không chuẩn hóa dữ liệu, các đặc trưng có đơn vị lớn hơn sẽ chi phối kết quả PCA.
- 5. Nhạy cảm với nhiễu:
 - Nhiễu trong dữ liệu có thể ảnh hưởng đến các trục chính được chọn.

5. Ứng dụng thực tế

1. Xử lý hình ảnh:
 - PCA được sử dụng để giảm kích thước ảnh mà vẫn giữ được các đặc điểm quan trọng.
 - Ví dụ: Nén ảnh mà không làm giảm chất lượng đáng kể.
2. Phân tích dữ liệu lớn:
 - Tìm các mẫu ẩn trong dữ liệu có số lượng đặc trưng lớn, như phân tích gen hoặc tài chính.
3. Tiền xử lý dữ liệu:
 - PCA thường được dùng để giảm chiều trước khi áp dụng các thuật toán học máy, như K-means, SVM hoặc Logistic Regression.
4. Tìm mẫu trong dữ liệu:
 - Tìm các thành phần chính trong dữ liệu để xác định xu hướng hoặc cụm (clusters).
5. Nén dữ liệu:
 - PCA giúp nén dữ liệu mà vẫn giữ được tỷ lệ lớn thông tin ban đầu.
6. Hệ thống gợi ý:
 - Sử dụng PCA để giảm chiều dữ liệu người dùng và sản phẩm, làm nhẹ hệ thống khi phân tích hành vi người dùng.

6. Ví dụ minh họa

Ví dụ:

Một công ty phân tích dữ liệu cần giảm chiều bộ dữ liệu chứa 100 đặc trưng để tăng tốc độ xử lý mà không làm mất thông tin quan trọng.

- Bước 1: Chuẩn hóa dữ liệu để các đặc trưng có cùng tầm quan trọng.
- Bước 2: Áp dụng PCA, tìm các trục chính.
- Bước 3: Chọn 10 trục chính đầu tiên (giữ lại 95% phương sai).
- Kết quả: Bộ dữ liệu mới chỉ có 10 chiều, dễ xử lý hơn mà vẫn giữ lại thông tin quan trọng.

7. So sánh với các phương pháp khác

Tiêu chí	PCA	LDA	Kernel PCA	t-SNE
Mục đích chính	Giảm chiều	Giảm chiều và phân loại	Giảm chiều phi tuyến	Trực quan hóa dữ liệu

Tiêu chí	PCA	LDA	Kernel PCA	t-SNE
Tương quan tuyến tính	Có	Có	Không	Không
Diễn giải	Khó	Tốt hơn	Khó	Khó
Yêu cầu tài nguyên	Ít	Ít	Trung bình	Cao

⇒ PCA là một công cụ mạnh mẽ trong tiền xử lý dữ liệu, nhưng cần được sử dụng cẩn thận để tránh mất thông tin quan trọng.

Bagging & Boosting

1. Cách thức hoạt động

Bagging là một kỹ thuật học máy ensemble nhằm cải thiện độ chính xác của mô hình bằng cách:

1. Tạo nhiều tập dữ liệu con từ tập dữ liệu gốc thông qua phương pháp bootstrap (lấy mẫu ngẫu nhiên có hoàn lại).
2. Xây dựng các mô hình độc lập (ví dụ: cây quyết định) trên từng tập dữ liệu con.
3. Kết hợp kết quả từ các mô hình bằng cách:
 - Hồi quy: Trung bình dự đoán từ các mô hình.
 - Phân loại: Lấy đa số phiếu từ các mô hình.

2. Mục đích

- Giảm phương sai (variance) của mô hình.
- Giúp mô hình hoạt động ổn định hơn và ít bị ảnh hưởng bởi dữ liệu nhiễu.

3. Điểm mạnh

- Hiệu quả với mô hình có độ phân tán cao (như cây quyết định).
- Dễ triển khai và có tính song song cao.
- Giảm nguy cơ overfitting vì các mô hình độc lập nhau.

4. Điểm yếu

- Không cải thiện nếu mô hình cơ bản có bias cao (mô hình đơn giản như hồi quy tuyến tính).
- Yêu cầu tài nguyên lớn khi số lượng mô hình tăng.

5. Ứng dụng

- Dự đoán tài chính: Tăng độ chính xác khi dự đoán giá cổ phiếu.
- Phân loại bệnh lý: Giảm sai số khi phân loại hình ảnh y khoa.

6. Ví dụ phổ biến

- Random Forest: Một biến thể của Bagging, sử dụng nhiều cây quyết định và kết hợp kết quả của chúng.

Boosting

1. Cách thức hoạt động

Boosting là một kỹ thuật học máy ensemble tập trung vào việc:

1. Xây dựng các mô hình liên tiếp, mỗi mô hình sau cải thiện lỗi của mô hình trước.
2. Trong mỗi bước, tăng trọng số cho các mẫu dữ liệu bị dự đoán sai, để mô hình sau tập trung hơn vào các lỗi này.
3. Kết hợp các mô hình cuối cùng bằng cách gán trọng số lớn hơn cho các mô hình có độ chính xác cao hơn.

2. Mục đích

- Giảm bias (độ thiên lệch) và cải thiện độ chính xác của mô hình.

3. Điểm mạnh

- Hiệu quả với dữ liệu phức tạp, phi tuyến.
- Tăng độ chính xác khi dự đoán nhờ tập trung vào lỗi.

4. Điểm yếu

- Nhạy cảm với nhiễu: Nếu dữ liệu có nhiễu nhiều, Boosting dễ bị overfitting.
- Tốn nhiều tài nguyên, đặc biệt khi số lượng mô hình tăng.

5. Ứng dụng

- Phát hiện gian lận: Tăng độ chính xác khi phân loại giao dịch gian lận.
- Nhận diện hình ảnh: Tăng hiệu suất trong các bài toán phân loại ảnh.

6. Ví dụ phổ biến

- AdaBoost: Một thuật toán Boosting cổ điển sử dụng các trọng số để cải thiện lỗi.
- Gradient Boosting: Cải thiện hiệu suất bằng cách tối ưu hóa hàm mất mát (loss function).
- XGBoost: Một phiên bản nâng cao của Gradient Boosting, nhanh hơn và hiệu quả hơn.

So sánh Bagging và Boosting

Tiêu chí	Bagging	Boosting
Mục tiêu chính	Giảm phương sai	Giảm bias
Cách xây dựng mô hình	Độc lập	Tuần tự, phụ thuộc vào lỗi mô hình trước
Xử lý overfitting	Giảm overfitting	Dễ bị overfitting nếu dữ liệu nhiễu
Hiệu quả với dữ liệu nhiễu	Tốt hơn	Nhạy cảm hơn
Tính song song	Cao	Thấp
Ví dụ phổ biến	Random Forest	AdaBoost, XGBoost

Ứng dụng thực tế của Bagging và Boosting

1. Bagging:
 - Dự đoán sản phẩm bán chạy trong thương mại điện tử.
 - Phân loại bệnh ung thư dựa trên dữ liệu y tế.
2. Boosting:
 - Phân loại giao dịch gian lận trong ngân hàng.
 - Dự đoán điểm tín dụng của khách hàng.

Hyperparameter

- Siêu tham số là những tham số được thiết lập trước quá trình huấn luyện và không được cập nhật trong quá trình học. Chúng ảnh hưởng trực tiếp đến cách mô hình học hỏi từ dữ liệu và hiệu suất tổng thể của nó. Một số ví dụ về siêu tham số là:
 - Learning rate: Tốc độ học của mô hình.
 - Batch size: Số lượng mẫu được sử dụng trong mỗi lần cập nhật trọng số.
 - Number of layers: Số lớp mạng trong mô hình.

- **Tree depth:** Độ sâu tối đa của cây quyết định trong các mô hình như Random Forest hoặc Gradient Boosting.
- Việc tìm ra giá trị tối ưu cho các siêu tham số này thường đòi hỏi các phương pháp tiếp cận có hệ thống để đảm bảo hiệu quả và tiết kiệm tài nguyên.

Tinh chỉnh siêu tham số (Hyperparameter Tuning)

- Tinh chỉnh siêu tham số là một bước quan trọng trong việc tối ưu hóa mô hình trí tuệ nhân tạo (AI) để đạt hiệu suất tốt nhất.

Phương pháp tinh chỉnh siêu tham số

1. Grid Search

- **Mô tả:** Grid Search thực hiện một cách tiếp cận toàn diện bằng cách liệt kê tất cả các kết hợp có thể của các giá trị siêu tham số trong một không gian xác định trước.
- **Cách thực hiện:**
 1. Xác định các giá trị có thể cho từng siêu tham số.
 2. Thử nghiệm từng tổ hợp một cách tuần tự.
 3. Đánh giá mô hình với mỗi tổ hợp và chọn tổ hợp có hiệu suất tốt nhất.
- **Ưu điểm:**
 - Đảm bảo kiểm tra toàn bộ không gian giá trị.
- **Hạn chế:**
 - Tốn thời gian và tài nguyên, đặc biệt khi không gian siêu tham số lớn.
- **Ứng dụng:** Phù hợp với không gian siêu tham số nhỏ hoặc khi số lượng tổ hợp có thể được kiểm tra là khả thi.

2. Random Search

- **Mô tả:** Random Search chọn ngẫu nhiên các giá trị siêu tham số từ một phạm vi xác định thay vì thử toàn bộ không gian.
- **Cách thực hiện:**
 1. Định nghĩa phạm vi giá trị cho mỗi siêu tham số.
 2. Chọn một tập hợp giá trị ngẫu nhiên và đánh giá mô hình.
- **Ưu điểm:**
 - Hiệu quả hơn Grid Search khi số lượng siêu tham số lớn.
 - Có thể đạt kết quả tốt với ít lần thử nghiệm hơn.
- **Hạn chế:**
 - Không đảm bảo kiểm tra toàn bộ không gian.
- **Ứng dụng:** Thường được sử dụng khi thời gian và tài nguyên hạn chế.

3. Bayesian Optimization

- **Mô tả:** Sử dụng phương pháp thống kê để ước tính hàm mục tiêu (thường là độ chính xác hoặc lỗi của mô hình) và tìm các giá trị siêu tham số có khả năng cải thiện hiệu suất.
- **Cách thực hiện:**

1. Xây dựng một mô hình xác suất để dự đoán hiệu suất dựa trên các thử nghiệm trước đó.
 2. Chọn các giá trị siêu tham số tiếp theo dựa trên mô hình này.
 3. Lặp lại cho đến khi đạt hiệu suất mong muốn.
- **Ưu điểm:**
 - Tối ưu hóa hiệu quả ngay cả khi không gian siêu tham số lớn.
 - **Hạn chế:**
 - Phức tạp hơn về mặt tính toán.
 - **Ứng dụng:** Thích hợp cho các bài toán phức tạp với nhiều siêu tham số.

4. Evolutionary Algorithms (Thuật toán tiến hóa)

- **Mô tả:** Dựa trên khái niệm tiến hóa tự nhiên, thuật toán tiến hóa khám phá không gian siêu tham số bằng cách tạo ra một tập hợp các giải pháp (quần thể) và cải thiện chúng qua các vòng lặp tiến hóa.
- **Cách thực hiện:**
 1. Tạo một quần thể các tổ hợp siêu tham số.
 2. Đánh giá hiệu suất và chọn ra các tổ hợp tốt nhất.
 3. Tạo ra thế hệ tiếp theo bằng cách biến đổi và kết hợp các tổ hợp tốt nhất.
 4. Lặp lại quá trình đến khi đạt được kết quả tốt nhất.
- **Ưu điểm:**
 - Tìm kiếm không gian phức tạp mà các phương pháp khác khó xử lý.
- **Hạn chế:**
 - Cần nhiều tài nguyên tính toán.
- **Ứng dụng:** Các bài toán tối ưu hóa phức tạp và không gian lớn.

5. Đánh giá chéo (Cross-validation) kết hợp tinh chỉnh siêu tham số

- **Mô tả:** Chia dữ liệu huấn luyện thành các tập con, sử dụng một tập để kiểm tra và các tập còn lại để huấn luyện. Lặp lại quá trình này để đánh giá hiệu suất tổng thể.
- **Cách thực hiện:**
 1. Chọn giá trị siêu tham số.
 2. Áp dụng Cross-validation để đánh giá hiệu suất của mỗi tổ hợp.
 3. Chọn tổ hợp có hiệu suất trung bình tốt nhất.
- **Ưu điểm:**
 - Đánh giá đáng tin cậy và giảm nguy cơ overfitting.
- **Hạn chế:**
 - Tốn thời gian khi dữ liệu lớn.
- **Ứng dụng:** Khi muốn đảm bảo mô hình có khả năng tổng quát hóa tốt.

Lưu ý khi tinh chỉnh siêu tham số

- **Cân nhắc tài nguyên:** Chọn phương pháp phù hợp với khả năng tính toán và thời gian.
- **Theo dõi kết quả:** Lưu trữ kết quả từng thử nghiệm để phân tích và so sánh.

- Kết hợp các phương pháp: Có thể bắt đầu với Random Search để thu hẹp không gian siêu tham số, sau đó sử dụng Bayesian Optimization hoặc Grid Search để tối ưu hóa.
- Tự động hóa: Sử dụng các thư viện như scikit-learn, Optuna, hoặc Hyperopt để hỗ trợ tinh chỉnh tự động.

Tinh chỉnh siêu tham số không chỉ nâng cao hiệu suất mô hình mà còn giúp hiểu rõ hơn về dữ liệu và tác động của từng tham số.

Chương 4: Unsupervised Learning

1. Phân cụm (Clustering)

- Phân cụm là một kỹ thuật trong Machine Learning không giám sát (Unsupervised Learning), nhằm xếp các đối tượng có điểm tương đồng vào cùng một cụm sao cho:

- Các đối tượng trong cùng một cụm: Càng giống nhau càng tốt.
- Các cụm khác nhau: Càng khác biệt nhau càng tốt.

Phân cụm được sử dụng trong nhiều lĩnh vực như: phân khúc khách hàng, phân tích gen, tự động nhận dạng hình ảnh, v.v.

2. Thuật toán phân cụm k-means (K-Means Clustering)

Quy trình:

1. Xác định số cụm k (cần xác định trước).
2. Chọn ngẫu nhiên k tâm cho k cụm.
3. Tính khoảng cách từ mỗi đối tượng đến tâm cụm.
4. Gộp đối tượng vào cụm mà khoảng cách tới tâm cụm là nhỏ nhất.
5. Cập nhật tâm cụm và lặp lại cho đến khi không có sự thay đổi.

3. Đo lường sự tương tự (Similarity Measure):

Khoảng cách thường được dùng:

- Khoảng cách Euclidean
- Khoảng cách Manhattan
- Khoảng cách Cosine

4. Phương pháp xác định số cụm tối ưu

1. Elbow Method

- Tính Within Cluster Sum of Squares (WSS) để đo độ lệch trong cụm.
- Vẽ biểu đồ WSS theo số cụm k.
- Tối ưu tại điểm góc khỷu (“elbow”), nơi mà tốc độ giảm WSS chậm lại.

2. Silhouette Score

- Đánh giá chất lượng phân cụm dựa trên sự gần gũi trong cùng cụm và sự khác biệt với các cụm khác.
- Silhouette score dao động từ -1 (đối tượng được phân vào nhóm sai) đến 1 (đối tượng được phân đúng nhóm).

5. Hierarchical Clustering

- Là một phương pháp phân nhóm (clustering) trong học máy, dùng để nhóm các đối tượng (data points) thành các nhóm (clusters) dựa trên mức độ tương tự lẫn nhau. Phương pháp này tạo ra một cây phân cấp (dendrogram), mô tả cách mà các đối tượng được nhóm lại với nhau theo các mức độ khác nhau.

Quy trình thực hiện:

1. Agglomerative (từ dưới lên)

Bước 1: Bắt đầu với mỗi điểm dữ liệu là một cụm riêng biệt

Bước 2: Gộp các cụm gần nhau nhất dựa trên 1 tiêu chí khoảng cách

Bước 3: Lặp lại quá trình gộp cho đến khi chỉ còn một cụm duy nhất

2. Divisive (từ trên xuống)

Bước 1: Bắt đầu với toàn bộ dữ liệu trong một cụm

Bước 2: Tách cụm lớn thành các cụm con cho đến khi mỗi điểm là một cụm riêng biệt.

Khi áp dụng **hierarchical clustering**, việc chọn giữa phương pháp **bottom-up (agglomerative)** và **top-down (divisive)** phụ thuộc vào mục tiêu và đặc điểm của dữ liệu bạn đang xử lý:

1. Đi từ dưới lên (Agglomerative - Bottom-up):

- Phương pháp này thường được sử dụng phổ biến hơn.
- Nó bắt đầu với mỗi đối tượng là một nhóm riêng biệt và dần dần hợp nhất các nhóm lại với nhau dựa trên độ tương tự cho đến khi tất cả các đối tượng thuộc về một nhóm duy nhất.
- **Ưu điểm:**
 - Phương pháp này rất linh hoạt và dễ triển khai.
 - Thường cho kết quả tốt khi bạn không biết trước số nhóm cần phân chia.
 - Quá trình tính toán có thể dễ dàng song song hóa.
- **Nhược điểm:**
 - Đôi khi mất nhiều thời gian khi làm việc với dữ liệu lớn, vì phải tính toán tất cả các khoảng cách giữa các nhóm.

2. Đi từ trên xuống (Divisive - Top-down):

- Phương pháp này bắt đầu với tất cả các đối tượng trong một nhóm duy nhất và sau đó chia nhỏ nhóm đó ra thành các nhóm con.
- **Ưu điểm:**
 - Có thể mang lại kết quả chính xác hơn trong một số trường hợp, nhất là khi bạn có thể đoán trước số lượng nhóm.
 - Thường tạo ra các nhóm có sự phân biệt rõ ràng hơn.
- **Nhược điểm:**

- Quá trình tính toán phức tạp và tốn thời gian hơn so với phương pháp bottom-up, vì phải tính toán phân chia nhóm liên tục từ đầu.
- Khó triển khai và ít được sử dụng trong thực tế.

Tóm lại, phương pháp **bottom-up (agglomerative)** thường là lựa chọn phổ biến và dễ triển khai hơn, đặc biệt khi không biết số nhóm cụ thể cần phân chia. Tuy nhiên, nếu bạn có thông tin rõ ràng về số lượng nhóm hoặc dữ liệu yêu cầu sự phân chia phức tạp, **top-down (divisive)** có thể sẽ là một lựa chọn tốt hơn.

Tiêu chí	K-means Clustering	Hierarchical Clustering
Số lượng cụm	Cần xác định trước k.	Không cần xác định trước.
Phương pháp phân cụm	Dựa trên khoảng cách đến tâm cụm.	Dựa trên khoảng cách giữa các điểm hoặc cụm.
Hiệu quả tính toán	Nhanh hơn, phù hợp với dữ liệu lớn.	Chậm hơn, khó áp dụng với dữ liệu lớn.
Hình dạng cụm	Phù hợp với cụm hình cầu, kích thước tương đồng.	Phù hợp với cụm phức tạp, không yêu cầu hình dạng cụ thể.
Trực quan hóa	Không cung cấp dendrogram.	Dendrogram thể hiện rõ mối quan hệ phân cụm.
Nhạy cảm với noise	Nhạy cảm với noise và outlier.	Ít nhạy cảm hơn nếu chọn khoảng cách phù hợp.

6. K-mode

K-mode là một thuật toán phân nhóm (clustering) trong học máy không giám sát (unsupervised learning), được phát triển để xử lý dữ liệu phân loại (categorical data). Nó là một biến thể của thuật toán K-means, nhưng thay vì sử dụng các trung bình (mean) như trong K-means, K-mode sử dụng chế độ (mode) để xác định trung tâm của mỗi nhóm.

Cách thức hoạt động:

- K-means hoạt động tốt với dữ liệu số (numerical data) bằng cách tính toán trung bình của các giá trị trong mỗi nhóm để cập nhật trung tâm nhóm.
- Tuy nhiên, với dữ liệu phân loại, việc tính trung bình không có ý nghĩa. Do đó, K-mode thay thế trung bình bằng mode (giá trị xuất hiện nhiều nhất trong một nhóm) để đại diện cho trung tâm của nhóm.

Quy trình K-mode:

1. Khởi tạo: Chọn k trung tâm nhóm ban đầu, mỗi trung tâm là một tập hợp giá trị phân loại (mode).

2. Gán đối tượng: Đối với mỗi đối tượng dữ liệu, gán nó vào nhóm có trung tâm mode gần nhất (dựa trên khoảng cách tính theo số lượng sự khác biệt giữa các thuộc tính).
3. Cập nhật trung tâm nhóm: Sau khi gán tất cả các đối tượng vào các nhóm, tính toán mode của các thuộc tính trong mỗi nhóm và cập nhật lại trung tâm nhóm.
4. Lặp lại: Lặp lại các bước trên cho đến khi trung tâm nhóm không thay đổi hoặc đạt tới một số vòng lặp cố định.

Đặc điểm của K-mode:

- Dữ liệu phân loại: K-mode được thiết kế đặc biệt cho dữ liệu phân loại, nơi các thuộc tính không có giá trị liên tục mà chỉ có các loại hoặc nhãn riêng biệt (ví dụ: giới tính, màu sắc, loại sản phẩm).
- Khoảng cách hamming: Để đo khoảng cách giữa các đối tượng trong K-mode, thay vì sử dụng khoảng cách Euclidean như trong K-means, K-mode sử dụng Khoảng cách Hamming (số lượng vị trí thuộc tính không trùng nhau giữa các đối tượng).

Ưu điểm của K-mode:

- Hiệu quả khi làm việc với dữ liệu phân loại, nơi không thể sử dụng các phương pháp như K-means.
- Có thể phân nhóm các đối tượng với các thuộc tính không liên tục, như phân nhóm khách hàng theo các thuộc tính như "Giới tính", "Tình trạng hôn nhân", v.v.

Nhược điểm:

- K-mode cũng cần số nhóm k được xác định trước, giống như K-means, điều này có thể gây khó khăn nếu không biết trước số nhóm hợp lý.
- Kết quả có thể phụ thuộc vào việc chọn giá trị k ban đầu, giống như K-means.

Ứng dụng:

- Phân nhóm khách hàng trong các bài toán marketing, ví dụ phân nhóm khách hàng dựa trên các đặc điểm phân loại như độ tuổi, giới tính, tình trạng hôn nhân.
- Phân nhóm các sản phẩm, dịch vụ dựa trên các thuộc tính phân loại.

7. Luật kết hợp

Luật kết hợp (association rule) là một khái niệm quan trọng trong khai thác dữ liệu, đặc biệt trong việc tìm kiếm mối quan hệ giữa các item trong cơ sở dữ liệu giao dịch. Mục tiêu của luật kết hợp là phát hiện các mối liên hệ hoặc mẫu trong dữ liệu, ví dụ như "Nếu khách hàng mua sản phẩm A, thì họ có khả năng cũng sẽ mua sản phẩm B."

Cấu trúc của một luật kết hợp:

Một luật kết hợp có dạng:

$A \rightarrow B$

Trong đó:

- A và B là các tập item (itemset) trong cơ sở dữ liệu.
- A là phần antecedent (tiền điều kiện) của luật.

- B là phần consequent (hậu điều kiện) của luật.

Ví dụ:

- $\{\text{sữa}\} \rightarrow \{\text{bánh mì}\}$ có nghĩa là "Nếu một khách hàng mua sữa, họ có thể sẽ mua bánh mì."

Các thành phần của luật kết hợp:

1. **Support (Hỗ trợ):**

- Là tỷ lệ giao dịch trong cơ sở dữ liệu chứa cả hai tập item A và B.
- Support đo lường mức độ phổ biến của một luật trong cơ sở dữ liệu.

Công thức tính support cho một luật $A \rightarrow B$:

$$\text{Support}(A \rightarrow B) = \frac{\text{Số lượng giao dịch chứa cả } A \text{ và } B}{\text{Tổng số giao dịch}}$$

Ví dụ: Nếu có 10 giao dịch và 2 trong số đó chứa cả {sữa, bánh mì}, thì support của $\{\text{sữa}\} \rightarrow \{\text{bánh mì}\}$ là $2/10 = 0.2$.

2. **Confidence (Độ tin cậy):**

- Là xác suất xảy ra B khi biết rằng A đã xảy ra. Nó đo lường mức độ chắc chắn của việc kết hợp giữa các item trong luật.

Công thức tính confidence cho một luật $A \rightarrow B$:

$$\text{Confidence}(A \rightarrow B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$$

Ví dụ: Nếu trong 100 giao dịch có 80 giao dịch chứa {sữa}, và trong số đó có 60 giao dịch chứa cả {sữa, bánh mì}, thì confidence của luật $\{\text{sữa}\} \rightarrow \{\text{bánh mì}\}$ là $60/80 = 0.75$.

3. **Lift (Lực hút):**

- Đo lường sự phụ thuộc giữa A và B. Lift cho thấy mối quan hệ giữa các item có mạnh mẽ hơn so với việc chúng xuất hiện độc lập.
- Nếu $\text{Lift} > 1$, điều đó có nghĩa là A và B có sự liên kết mạnh mẽ hơn so với việc chúng xuất hiện ngẫu nhiên.

Công thức tính lift cho một luật $A \rightarrow B$:

$$\text{Lift}(A \rightarrow B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A) \times \text{Support}(B)}$$

Ví dụ: Nếu support của {sữa, bánh mì} là 0.2, support của {sữa} là 0.4, và support của {bánh mì} là 0.3, thì lift của luật $\{\text{sữa}\} \rightarrow \{\text{bánh mì}\}$ là $0.2 / (0.4 \times 0.3) = 1.67$.

Ví dụ về luật kết hợp:

Giả sử cơ sở dữ liệu của bạn có các giao dịch sau:

Giao dịch Mặt hàng

- | | |
|---|----------------|
| 1 | {sữa, bánh mì} |
| 2 | {sữa, bơ} |
| 3 | {sữa, bánh mì} |
| 4 | {bánh mì, bơ} |
| 5 | {sữa, bánh mì} |

Từ bảng dữ liệu này, bạn có thể rút ra một số luật kết hợp như sau:

1. {sữa} $\rightarrow$ {bánh mì}
 - Support: $3/5 = 0.6$ (60% giao dịch có chứa cả sữa và bánh mì).
 - Confidence: $3/4 = 0.75$ (75% giao dịch có chứa sữa cũng có bánh mì).
 - Lift: $0.6 / (0.8 * 0.8) = 0.9375$ (chỉ ra rằng sữa và bánh mì xuất hiện cùng nhau ít hơn mong đợi nếu chúng độc lập).
2. {bánh mì} $\rightarrow$ {sữa}
 - Support: $3/5 = 0.6$
 - Confidence: $3/4 = 0.75$
 - Lift: $0.6 / (0.8 * 0.8) = 0.9375$

Các ứng dụng của luật kết hợp:

1. Marketing: Tìm ra các mối quan hệ giữa các sản phẩm để đưa ra chiến lược bán hàng hoặc khuyến mãi. Ví dụ: Nếu khách hàng mua một sản phẩm, họ có thể sẽ mua sản phẩm khác (như "mua sữa thì cũng sẽ mua bánh mì").
2. Giới thiệu sản phẩm: Sử dụng các luật kết hợp để xây dựng hệ thống gợi ý sản phẩm cho người dùng, ví dụ như trong các cửa hàng trực tuyến.
3. Phân tích hành vi khách hàng: Khám phá các mẫu hành vi trong dữ liệu giao dịch của khách hàng để đưa ra các quyết định kinh doanh thông minh.

Tóm lại:

- Luật kết hợp là một công cụ mạnh mẽ để phát hiện các mối quan hệ giữa các item trong một tập dữ liệu lớn, đặc biệt là trong các lĩnh vực như bán lẻ, marketing, và hệ thống gợi ý.
- Việc tính toán các giá trị như support, confidence, và lift giúp đánh giá mức độ mạnh mẽ và đáng tin cậy của các mối quan hệ giữa các item trong cơ sở dữ liệu.

8. Apriori Algorithm

Thuật toán Apriori là một thuật toán phổ biến trong khai thác luật kết hợp (association rule mining), được sử dụng để phát hiện các mẫu hoặc mối quan hệ trong dữ liệu, đặc biệt là trong các tập dữ liệu lớn như các giao dịch mua bán trong siêu thị. Apriori được phát triển bởi R. Agrawal và R. Srikant vào năm 1994.

Mục tiêu của Apriori:

Thuật toán Apriori nhằm tìm kiếm các tập item thường xuyên (frequent itemsets) trong cơ sở dữ liệu giao dịch, sau đó sử dụng chúng để tạo ra các luật kết hợp (association rules). Các luật này có thể giúp chúng ta hiểu mối quan hệ giữa các mặt hàng trong các giao dịch.

Ví dụ: Nếu khách hàng mua sữa và bánh mì, họ có thể có xu hướng mua bơ. Đó là một ví dụ của một luật kết hợp.

Các khái niệm cơ bản trong Apriori:

1. Itemset: Một tập hợp các mặt hàng trong một giao dịch. Ví dụ: {sữa, bánh mì}.
2. Frequent Itemset: Một itemset được gọi là "thường xuyên" nếu nó xuất hiện trong ít nhất một phần trăm các giao dịch trong cơ sở dữ liệu, gọi là minimum support.
3. Support: Tỷ lệ của các giao dịch chứa một itemset cụ thể. Ví dụ, nếu 100 giao dịch có 10 giao dịch chứa {sữa, bánh mì}, thì support của itemset {sữa, bánh mì} là 10%.
4. Confidence: Đo lường độ tin cậy của một luật kết hợp. Ví dụ, nếu có 80% của những người mua {sữa, bánh mì} cũng mua bơ, thì độ tin cậy của luật {sữa, bánh mì} $\rightarrow$ {bơ} là 80%.
5. Lift: Đo lường sự phụ thuộc giữa các item trong luật kết hợp. Nếu lift lớn hơn 1, thì các item trong luật kết hợp có xu hướng xuất hiện cùng nhau.
6. Transaction Database: cơ sở dữ liệu lưu trữ các giao dịch mua bán của khách hàng.
7. Transaction: một giao dịch cụ thể, chứa một hoặc nhiều mục sản phẩm được khách hàng mua.

Quy trình hoạt động của thuật toán Apriori:

Thuật toán Apriori hoạt động theo nguyên lý lặp đi lặp lại. Dưới đây là các bước chi tiết:

1. Tìm các itemset thường xuyên bậc 1 (1-itemsets):

- Đầu tiên, thuật toán sẽ tính toán support của tất cả các item đơn lẻ trong tập dữ liệu (các itemsets bậc 1). Mỗi itemset được so sánh với minimum support threshold để xem nó có phải là frequent itemset hay không.

2. Tạo các itemset thường xuyên bậc 2 (2-itemsets):

- Sau khi có các itemsets bậc 1, thuật toán sẽ tạo ra tất cả các itemsets bậc 2 từ các itemset bậc 1 (kết hợp các itemset bậc 1 với nhau). Sau đó, tính support cho các itemset bậc 2 và chỉ giữ lại những itemsets có support lớn hơn hoặc bằng minimum support.

3. Lặp lại cho đến khi không còn itemset thường xuyên:

- Thuật toán sẽ tiếp tục tăng bậc của itemsets (từ bậc 3, bậc 4, ...) và lặp lại bước trên cho đến khi không thể tạo ra thêm bất kỳ itemset thường xuyên nào.

4. Tạo luật kết hợp:

- Khi các itemset thường xuyên được xác định, thuật toán sẽ tạo ra các luật kết hợp từ các itemset này. Một luật kết hợp có dạng $A \rightarrow B$, nghĩa là nếu A xảy ra, thì B có khả năng xảy ra. Để tạo ra luật, thuật toán sẽ tính confidence và lift của mỗi luật.
- Nếu confidence và lift của một luật đạt ngưỡng nhất định, nó sẽ được giữ lại.

Ví dụ minh họa:

Giả sử có một cơ sở dữ liệu giao dịch như sau:

Giao dịch Mặt hàng

1	{sữa, bánh mì}
2	{sữa, bơ}
3	{sữa, bánh mì}
4	{bánh mì, bơ}
5	{sữa, bánh mì}

Bước 1: Tính toán support của các item đơn lẻ (itemsets bậc 1):

- $\text{Support}(\text{sữa}) = 4/5 = 0.8$
- $\text{Support}(\text{bánh mì}) = 4/5 = 0.8$
- $\text{Support}(\text{bơ}) = 3/5 = 0.6$

Bước 2: Tạo các itemset bậc 2:

- $\text{Support}(\{\text{sữa, bánh mì}\}) = 3/5 = 0.6$ (frequent itemset nếu minimum support ≥ 0.5)
- $\text{Support}(\{\text{sữa, bơ}\}) = 2/5 = 0.4$ (không phải là frequent itemset nếu minimum support ≥ 0.5)
- $\text{Support}(\{\text{bánh mì, bơ}\}) = 2/5 = 0.4$ (không phải là frequent itemset nếu minimum support ≥ 0.5)

Bước 3: Tạo luật kết hợp từ itemset {sữa, bánh mì}:

- $\text{Confidence}(\{\text{sữa}\} \rightarrow \{\text{bánh mì}\}) = \text{Support}(\{\text{sữa, bánh mì}\}) / \text{Support}(\{\text{sữa}\}) = 0.6 / 0.8 = 0.75$
- $\text{Confidence}(\{\text{bánh mì}\} \rightarrow \{\text{sữa}\}) = \text{Support}(\{\text{sữa, bánh mì}\}) / \text{Support}(\{\text{bánh mì}\}) = 0.6 / 0.8 = 0.75$

Các luật kết hợp có thể được kiểm tra thêm với lift, nhưng ở đây ta chỉ tính confidence.

Ưu điểm của Apriori:

- Hiệu quả trong tìm kiếm luật kết hợp: Apriori có thể giúp phát hiện các mối quan hệ tiềm ẩn giữa các item trong cơ sở dữ liệu.
- Dễ hiểu và dễ triển khai: Phương pháp này đơn giản và dễ áp dụng cho các bài toán khai thác dữ liệu.

Nhược điểm của Apriori:

- Tốn kém về mặt tính toán: Với các tập dữ liệu lớn, Apriori có thể gặp khó khăn trong việc tính toán vì phải tạo ra nhiều itemsets và kiểm tra tất cả các kết hợp.
- Quá trình lặp lại: Thuật toán cần phải lặp đi lặp lại nhiều lần cho đến khi không còn itemsets nào thỏa mãn ngưỡng hỗ trợ, điều này có thể làm cho thời gian tính toán dài.
- Không tối ưu cho dữ liệu dày đặc: Apriori có thể không hiệu quả trong các dữ liệu quá lớn hoặc quá dày đặc, vì nó phải kiểm tra rất nhiều kết hợp.

Ứng dụng của Apriori:

- Marketing: Phân tích các giao dịch để tìm ra các mặt hàng thường xuyên xuất hiện cùng nhau, từ đó tạo ra các chiến lược marketing và khuyến mãi hiệu quả.
- Phân tích giỏ hàng: Phát hiện các mặt hàng thường xuyên được mua cùng nhau trong các siêu thị hoặc cửa hàng trực tuyến.
- Phân tích hành vi người dùng: Xác định các hành vi của người dùng trong các hệ thống khuyến nghị sản phẩm.

Chương 5: Neural Network

1. Định nghĩa mạng nơ-ron nhân tạo?

- Mạng nơ-ron nhân tạo là mô hình học sâu lấy cảm hứng từ cấu trúc não của con người. Mạng Neural được sử dụng để xử lý các nhiệm vụ phức tạp như nhận diện hình ảnh, xử lý ngôn ngữ tự nhiên,... và nhiều ứng dụng khác trong lĩnh vực trí tuệ nhân tạo và học máy.

2. Các thành phần của mạng nơ-ron nhân tạo

- **Neuron**: là đơn vị cơ bản, nó như một tế bào thần kinh. Mỗi nơ-ron nhận đầu vào, tính toán và truyền đầu ra.
- **Layer**: Các lớp cấu thành mạng nơ-ron
 - **Input layer**: Nhận dữ liệu đầu vào
 - **Hidden layer**: Xử lý và trích xuất đặc trưng từ dữ liệu. Số lượng và độ phức tạp của hidden layer ảnh hưởng đến khả năng học của mạng.
 - **Output layer**: Trả về kết quả dự đoán
- **Weight**: trọng số, xác định mức độ quan trọng của mỗi đầu vào với đầu ra
- **Bias**: Giúp mạng dịch chuyển hàm kích hoạt và tăng khả năng học linh hoạt
- **Activation Function**: Quyết định các thức neuron chuyển đổi tín hiệu đầu vào thành đầu ra.

3. Weights, bias, các loại activation function

Weights

- Trọng số là tham số xác định mức độ quan trọng của đầu vào đối với mỗi đầu ra
- Mỗi kết nối giữa các nơ-ron đều được gán trọng số

Hoạt động:

- Tại mỗi nơ-ron, đầu vào xi được nhân với trọng số wi tương ứng, tổng hợp của tất cả đầu vào và trọng số là:

$$z = \sum_{i=1}^n w_i x_i$$

- Kết quả z này sẽ được đưa vào hàm kích hoạt để tạo đầu ra của neuron.

Bias

- Là một giá trị cố định được cộng vào tổng trọng số và đầu vào.
- Nó giúp cho mạng dịch chuyển hàm kích hoạt để đạt được đầu ra mong muốn
- Làm tăng tính linh hoạt của mô hình, giúp mạng nơ-ron học được được nhiều mối quan hệ phức tạp hơn.

Activation Function

- Hàm kích hoạt là một hàm toán học được áp dụng tại mỗi nơ-ron trong mạng nơ-ron để quyết định xem tín hiệu từ đầu vào có truyền được qua mạng hay không.
- Mục đích: Hàm kích hoạt đưa tính phi tuyến tính vào mạng nơ-ron, cho phép mạng học các mối quan hệ phức tạp và phi tuyến tính trong dữ liệu.

- Nếu không có hàm kích hoạt thì mạng nơ ron sẽ chỉ là một mô hình tuyến tính, nó không thể giải quyết những bài toán phức tạp
- Nó cũng giúp giới hạn đầu ra trong một khoảng nhất định ví dụ $[0,1]$ hoặc $[-1,1]$ giúp cho mạng ổn định hơn trong quá trình huấn luyện

Các loại Activation Function

1. Sign (Perceptron)

- Công thức:

$$f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

- Đặc điểm:
 - Dạng đầu ra: $\{-1,1\}$
 - Tính chất: Đây là hàm phân loại tuyến tính, thường dùng trong Perceptron ban đầu

Ưu điểm

- Đơn giản nhanh chóng
- Phù hợp cho bài toán phân loại đơn giản

Nhược điểm

- Không dùng được với Gradient Descent
- Không hỗ trợ phi tuyến tính, hạn chế với những bài toán phức tạp

⇒ Ít được sử dụng trong các bài toán thực tế hiện nay vì không hỗ trợ các thuật toán tối ưu hiện đại

Sigmoid

Công thức:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Đặc điểm:

- Dạng đầu ra: $(0,1)$
- Gradient: $f'(x) = f(x)(1-f(x))$
- Tăng dần khi $x \rightarrow \infty$, giảm dần khi $x \rightarrow -\infty$

Ưu điểm:

- Dễ tính toán và mượt mà do đó nó rất phổ biến trong các mô hình nơ ron cổ điển
- Kết quả đầu ra từ 0 đến 1 làm cho hàm sigmoid phù hợp để biểu diễn xác suất

Nhược điểm:

- Bị Vanishing Gradient: Khi giá trị quá lớn hoặc quá nhỏ, sẽ khiến cho gradient gần bằng 0 khiến cho mô hình học chậm thậm chí là không học được.

Tanh

Công thức

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Đặc điểm:

- Dạng đầu ra: $(-1, 1)$
- Gradient: $f'(x) = 1 - f(x)^2$
- Trung tâm tại 0

Ưu điểm

- Giá trị đầu ra cân bằng hơn sigmoid
- Phù hợp cho dữ liệu có giá trị âm và dương

Nhược điểm:

- Gradient Vanishing

ReLU

Công thức:

$$f(x) = \max(0, x)$$

Đặc điểm:

- Dạng đầu ra: $[0, \infty)$.
- Gradient: $f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$.

Ưu điểm:

- Đơn giản, tính toán nhanh
- Không gặp vấn đề về gradient vanishing khi $x > 0$

Nhược điểm:

- Dying ReLU: Neuron có thể chết (luôn đưa về 0)

Leaky ReLU

Công thức:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$$

Với α là một hằng số nhỏ (thường là 0.01).

Đặc điểm giống ReLU nhưng xử lý được vấn đề Dying ReLU

Ưu điểm

- Không có neuron chết hoàn toàn

Nhược điểm

- Cần chọn giá trị alpha cho phù hợp

Softmax: trả về 1 phân phối xác suất của từng lớp này

Công thức

$$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Đặc điểm:

- Dạng đầu ra: Phân phối xác suất (tổng các giá trị bằng 1)
- Tăng cường khác biệt giữa các lớp

Ưu điểm:

- Phù hợp với bài toán phân loại đa lớp
- Đầu ra dễ diễn giải xác suất

Nhược điểm

- Tốn tính toán hơn các hàm khác

Softplus

Công thức:

$$f(x) = \ln(1 + e^x)$$

Đặc điểm:

- Giống ReLU nhưng mượt hơn
- Gradient không bằng 0 tại bất kỳ điểm nào

Ưu điểm:

- Xử lý gradient vanishing tốt hơn sigmoid
- Tron trư hơn ReLU

Nhược điểm

- Tốn nhiều tính toán hơn ReLU

ELU

Công thức:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x \leq 0 \end{cases}$$

Đặc điểm:

* Khi $x \leq 0$, ELU giảm dần xuống giá trị âm thay vì về 0 (như ReLU).

Ưu điểm:

- Trợ giúp gradient trong vùng giá trị âm
- Giải quyết vấn đề Dying ReLU

Nhược điểm:

- Cần chọn tham số alpha
- Tốn tính toán hơn ReLU

So sánh các hàm kích hoạt

Hàm	Ưu điểm	Nhược điểm	Khi nào dùng
Sign	Đơn giản	Không khả vi	Ít dùng trong thực tế.
Sigmoid	Xác suất, dễ diễn giải	Gradient vanishing	Phân loại nhị phân.
Tanh	Cân bằng hơn sigmoid	Gradient vanishing	Khi đầu ra cần cân bằng $(-1, 1)$.
ReLU	Đơn giản, hiệu quả	Dying ReLU	Phổ biến nhất cho mạng sâu.
Leaky ReLU	Giải quyết Dying ReLU	Cần chọn α	Khi gặp vấn đề Dying ReLU.
Softmax	Xác suất đa lớp	Tốn tính toán	Phân loại đa lớp.
Softplus	Trơn tru hơn ReLU	Tốn tính toán	Thay thế ReLU khi cần trơn tru.
ELU	Gradient ổn định hơn trong vùng âm	↓ Cần chọn α , tốn tính toán	Khi gặp gradient vanishing.

1. Mạng nơ-ron đơn lớp

Định nghĩa:

- Mạng nơ-ron đơn lớp là một mạng có cấu trúc bao gồm **một lớp đầu vào** (input layer) và **một lớp đầu ra** (output layer), không có bất kỳ lớp ẩn nào ở giữa.
- Lớp đầu ra kết nối trực tiếp với đầu vào thông qua các trọng số (weights) và hàm kích hoạt (activation function).

Cấu trúc:

- Input Layer: Nhận dữ liệu từ bên ngoài (số lượng nơ-ron trong lớp này tương ứng với số đặc trưng của dữ liệu).
- Output Layer: Trả kết quả của mạng, ví dụ:
 - Với bài toán phân loại, đầu ra thường là xác suất của các lớp.
 - Với bài toán hồi quy, đầu ra là giá trị thực.

Ưu điểm:

- Đơn giản, dễ hiểu và tính toán nhanh.
- Phù hợp với các bài toán đơn giản như hồi quy tuyến tính hoặc phân loại nhị phân.

Nhược điểm:

- Không thể xử lý các bài toán phức tạp hoặc phi tuyến tính.
- Khả năng biểu diễn bị hạn chế do không có lớp ẩn để học các biểu diễn trừu tượng hơn.

2. Mạng nơ-ron đa lớp

Định nghĩa:

- Mạng nơ-ron đa lớp (thường được gọi là MLP - Multi-layer Perceptron) là một mạng nơ-ron có ít nhất **một lớp ẩn** (hidden layer) giữa lớp đầu vào và lớp đầu ra.
- Mỗi lớp ẩn sử dụng các hàm kích hoạt để học các mối quan hệ phi tuyến tính.

Cấu trúc:

- **Input Layer:** Nhận dữ liệu đầu vào.
- **Hidden Layers:** Một hoặc nhiều lớp ẩn, nơi mạng học các biểu diễn trừu tượng từ dữ liệu thông qua các trọng số và hàm kích hoạt phi tuyến (ví dụ: ReLU, sigmoid, tanh).
- **Output Layer:** Tạo ra kết quả cuối cùng.

Ưu điểm:

- Có khả năng học và biểu diễn các mối quan hệ phức tạp, bao gồm cả những mối quan hệ phi tuyến tính.
- Hiệu quả cho nhiều loại bài toán như phân loại, hồi quy, nhận dạng ảnh, xử lý ngôn ngữ tự nhiên.
- Độ chính xác cao hơn mạng đơn lớp khi được tối ưu hóa tốt.

Nhược điểm:

- Tốn kém tài nguyên tính toán, đặc biệt khi có nhiều lớp và nhiều nơ-ron.
- Yêu cầu kỹ thuật tối ưu hóa tốt (như chuẩn hóa, điều chỉnh tham số) để tránh hiện tượng **overfitting** hoặc **vanishing gradient**.
- Cần nhiều dữ liệu để đạt hiệu quả cao.

1. Perceptron

- Mạng nơ ron đơn giản nhất, chỉ có 1 input và output layer, chỉ sử dụng để giải quyết những bài toán phân loại tuyến tính
 - **Các thành phần chính**
 - + Input (các đặc trưng)
 - + Bias (hệ số điều chỉnh hàm kích hoạt, điều chỉnh sao cho đường phân lớp đồ)
 - + Weights (Trọng số quyết định mức độ quan trọng của đặc trưng)
 - + Sum Function (Hàm tổng là tổng trọng số của input)
 - + Activate Function
 - **Quy trình hoạt động**
 1. Nhận dữ liệu đầu vào
 2. Tính tổng trọng số
 3. Áp dụng hàm kích hoạt
 4. Cập nhật trọng số và bias (nếu sai số tồn tại)
 5. Lặp lại cho đến khi hội tụ (cho đến khi độ chính xác lớn, hay số vòng lặp đủ lớn)
 - **Quy trình học**
 - Mục tiêu quá trình học là điều chỉnh các trọng số sao cho mô hình có thể dự đoán được tốt nhất, độ chính xác cao.
 - Lan truyền tiến
 - Lan truyền ngược
- ⇒ Kết quả cuối cùng: In ra trọng số bằng bao nhiêu

4. Ưu và Nhược điểm của Mạng Perceptron

Ưu điểm:

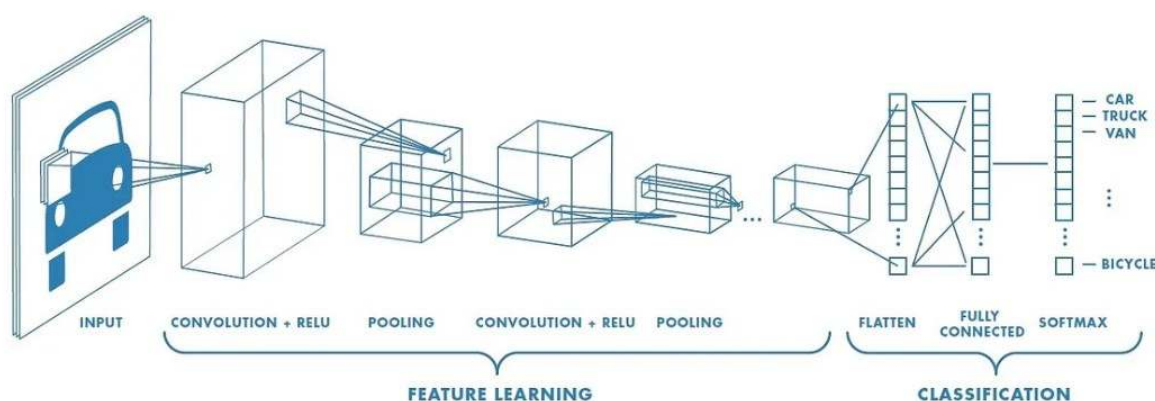
1. Đơn giản: Cấu trúc và thuật toán học dễ hiểu, dễ triển khai.
2. Hiệu quả với bài toán tuyến tính: Phân loại dữ liệu có thể được phân tách tuyến tính.
3. Nhanh chóng: Do không quá phức tạp, Perceptron xử lý nhanh với dữ liệu nhỏ.

Nhược điểm:

1. Chỉ xử lý được bài toán tuyến tính:
 - Không thể phân loại dữ liệu phi tuyến tính (ví dụ: bài toán XOR).
2. Không khả vi tại một số điểm:
 - Hàm kích hoạt Sign không khả vi, gây khó khăn trong các bài toán tối ưu hiện đại.
3. Nhạy cảm với lựa chọn Learning Rate:
 - Learning rate quá lớn có thể gây mất hội tụ, quá nhỏ thì học chậm.

1. CNN

- Mạng nơ ron tích chập là một loại mạng nơ ron nhân tạo đặc biệt, được thiết kế để xử lý và phân tích dữ liệu có cấu trúc dạng lưới, chẳng hạn như hình ảnh. CNN thường được dùng trong các bài toán về thị giác máy tính như nhận diện ảnh, phân loại ảnh.



Kiến trúc của CNN

1. Input layer

- Lớp tiếp nhận dữ liệu đầu vào, thường là hình ảnh hoặc dữ liệu được biểu diễn dưới dạng ma trận

2. Convolutional layer

- Đây là lớp chính của mạng CNN, nơi mà các kernel (filter) được sử dụng để trích xuất các đặc trưng
- Kernel là một ma trận nhỏ có kích thước cố định và nó sẽ di chuyển trên toàn bộ hình ảnh đầu vào.
- Output của lớp này là feature map, đại diện cho các lớp đặc trưng được trích xuất từ dữ liệu

3. Activation layer

- Sau khi thực hiện lớp tích chập, một hàm kích hoạt (thường là ReLU) được áp dụng để đưa đầu ra về miền giá trị không âm, tăng khả năng học phi tuyến.

4. Pooling Layer

- Lớp này sẽ được dùng để giảm kích thước của feature map, giúp giảm số lượng tham số và tăng tính hiệu quả tính toán
- Các loại gộp phổ biến:
 - Max pooling: Lấy giá trị lớn nhất trong vùng $n \times n$
 - Average Pooling: Lấy giá trị trung bình trong vùng $n \times n$

5. Normalization Layer

- Chuẩn hóa dữ liệu để tăng tốc độ huấn luyện và giảm khả năng bị overfitting
- Kỹ thuật phổ biến là Batch Normalization

6. Fully connected Layer

- Sau khi qua các lớp tích chập và gộp, đầu ra sẽ được làm phẳng thành 1 vector
- Vector này được đưa qua một hoặc nhiều lớp fully connected để thực hiện các tác vụ như phân loại hoặc dự đoán.

7. Output Layer

- Đây là lớp cuối cùng, thường sử dụng hàm Softmax để đưa ra xác suất cho mỗi lớp trong bài toán phân loại

Cách thức hoạt động của CNN

1. Input dữ liệu

Hình ảnh đầu vào được biểu diễn dưới dạng tensor $H \times W \times D$, trong đó:

- H: chiều cao (Height)
- W: chiều rộng (Width)
- D: số kênh (Depth), ví dụ: 3 đối với ảnh RGB.

2. Trích xuất đặc trưng

Các lớp tích chập đầu tiên trích xuất các đặc trưng cơ bản như cạnh, góc. Lớp sâu hơn sẽ học các đặc trưng phức tạp hơn như hình dạng, vật thể.

3. Giảm kích thước và tăng hiệu quả

Các lớp gộp giúp giảm kích thước của feature map và làm nổi bật các đặc trưng quan trọng.

4. Kết hợp đặc trưng

Lớp fully connected kết hợp các đặc trưng đã trích xuất để thực hiện nhiệm vụ như phân loại hoặc dự đoán.

5. Dự đoán đầu ra

Lớp Softmax (hoặc hàm kích hoạt khác) đưa ra kết quả cuối cùng.

Ưu điểm của CNN

1. **Hiệu quả với dữ liệu hình ảnh:** Trích xuất đặc trưng tự động và không cần kỹ thuật tiền xử lý phức tạp.
2. **Khả năng học không gian:** Học các đặc trưng cục bộ và kết hợp để tạo thành đặc trưng toàn cục.
3. **Giảm số tham số:** Kernel chia sẻ tham số, giảm độ phức tạp tính toán so với mạng fully connected.

2. Gradient Descent

3. Backward propagation

4. Deep learning

- Là một nhánh của **Machine Learning** trong đó các mô hình học từ dữ liệu thông qua việc sử dụng các **mạng nơ-ron nhân tạo** với nhiều tầng. Mục tiêu chính của Deep Learning là tự động trích xuất các đặc trưng phức tạp từ dữ liệu để giải quyết các bài toán như phân loại, dự đoán, nhận dạng, và nhiều ứng dụng khác.

1. Keras

2. GANs